

MODUL PELATIHAN

PELATIHAN REACT FUNDAMENTAL

Membangun UI Modern
dengan JavaScript Library



Komponen

Bangun UI dengan
komponen yang reusable



Hooks

Kelola state dan side effect
dengan lebih mudah



Efisien & Scalable

Bangun aplikasi yang cepat,
terstruktur, dan mudah dirawat



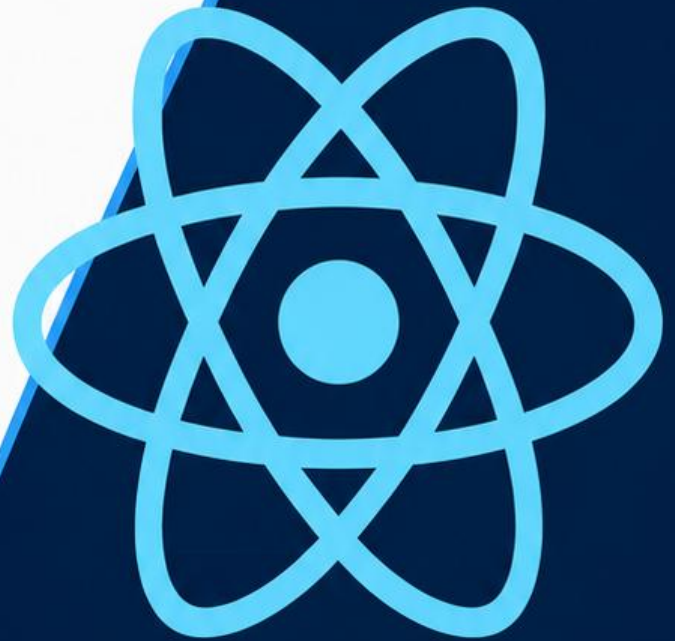
PEMBERDAYAAN
UMAT BERKELANJUTAN (PUB)



```
import React from 'react';

function App() {
  return (
    <div className="app">
      <h1=Hello React!</h1>
    </div>
  )
}

export default App
```



Untuk Pemula hingga
Menengah



Studi Praktik &
Contoh Proyek



Project Based
Learning

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga Modul Pelatihan React Fundamental ini dapat disusun dengan baik.

Modul ini dirancang sebagai panduan pembelajaran bagi peserta dalam mempelajari dasar-dasar pengembangan aplikasi web menggunakan React.js. Materi disusun secara bertahap, dimulai dari pengenalan JavaScript Modern (ES6), Node.js, npm, pnpm, dan Vite, kemudian dilanjutkan dengan konsep dasar React seperti komponen, JSX, props, state, conditional rendering, list rendering, hingga implementasi fitur filter, sorting, pagination, editing, Context API, React Router, serta deployment aplikasi ke Vercel. Sebagai evaluasi, modul ini juga dilengkapi dengan proyek UTS dan UAS yang mengintegrasikan seluruh materi yang telah dipelajari.

Setiap bab dalam modul ini dilengkapi dengan penjelasan konsep, contoh implementasi, dan latihan sederhana agar peserta dapat memahami materi secara bertahap dan mampu menerapkannya dalam pengembangan aplikasi React.

Penyusun menyadari bahwa modul ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan modul ini di masa mendatang. Semoga modul ini dapat memberikan manfaat dan menjadi referensi yang baik bagi peserta dalam mempelajari React Fundamental.

Jakarta, 04 Juli 2026

Wahyu

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENGENALAN JAVASCRIPT MODERN (ES6)	1
1.1. Pendahuluan.....	1
1.2. Variabel (let dan const)	1
1.3. Arrow Function	3
1.4. Template Literals	5
1.5. Destructuring Assignment.....	6
1.6. Spread dan Rest Operator	7
1.7. Ringkasan Bab.....	8
BAB 2 PENGENALAN NODE.JS, NPM DAN PNPM	10
2.1. Pendahuluan.....	10
2.2. Node.js.....	10
2.3. Pengenalan npm (Node Package Manager)	12
2.4. Pengenalan pnpm.....	15
2.5. Membuat dan Mengelola Project.....	17
2.6. Ringkasan Bab.....	18
BAB 3 MENGGUNAKAN PNPM DAN VITE.....	19
3.1. Pendahuluan.....	19
3.2. Mengenal Vite	20
3.3. Membuat Project.....	20
3.4. Struktur Folder Project.....	22
3.5. Mengenal File package.json	23

3.6.	Perintah Dasar pnpm.....	24
3.7.	Ringkasan Bab.....	25
BAB 4 DASAR-DASAR REACT DAN KOMPONEN.....		26
4.1.	Pendahuluan.....	26
4.2.	Apa itu React?	26
4.3.	Mengenal JSX	28
4.4.	Mengenal Komponen.....	30
4.5.	Membuat Komponen Pertama	31
4.6.	Menggunakan Komponen.....	32
4.7.	Ringkasan Bab.....	32
BAB 5 MENERUSKAN PROPS DAN STYLING KOMPONEN.....		34
5.1.	Pendahuluan.....	34
5.2.	Apa itu Props?	34
5.3.	Mengirim Props	35
5.4.	Menerima Props	36
5.5.	Styling React Menggunakan Tailwind CSS.....	36
5.6.	Instalasi Tailwind CSS.....	37
5.7.	Mengenal Utility Class Tailwind CSS	38
BAB 6 CONDITIONAL RENDERING DAN LIST RENDERING.....		40
6.1.	Pendahuluan.....	40
6.2.	Apa itu Conditional Rendering?	40
6.3.	Apa itu List Rendering?	42
6.4.	Studi Kasus	44
6.5.	Ringkasan Bab.....	46
BAB 7 MERESPONS EVENT DAN STATE MANAGEMENT		47

7.1.	Pendahuluan.....	47
7.2.	Apa itu Event Handling?.....	47
7.3.	Apa itu State?	48
7.4.	Studi Kasus	50
7.5.	Ringkasan Bab.....	51
BAB 8 UJIAN TENGAH SEMESTER (PROYEK SEDERHANA).....		52
BAB 9 FILTER & PENCARIAN.....		53
9.1.	Pendahuluan.....	53
9.2.	Apa Itu Pencarian	53
9.3.	Mengenal Method filter()	54
9.4.	filter() pada Data Produk	55
9.5.	Membuat Input Pencarian.....	56
9.6.	Menghubungkan Search dengan filter().....	57
9.7.	Menampilkan Hasil Pencarian.....	57
9.8.	Conditional Rendering pada Hasil Pencarian.....	58
9.9.	Ringkasan Bab.....	63
BAB 10 SORTING		64
10.1.	Pendahuluan.....	64
10.2.	Apa Itu Sorting?.....	64
10.3.	Mengenal sort().....	65
10.4.	Implementasi Sorting.....	65
10.5.	Ringkasan Bab.....	74
BAB 11 PAGINATION		75
11.1.	Pendahuluan.....	75
11.2.	Apa Itu Pagination?	76

11.3.	Pagination pada CareerHub	77
11.4.	Konsep Dasar Pagination	78
11.5.	Mengenal Math.ceil()	79
11.6.	Menentukan Data yang Ditampilkan.....	79
11.7.	Ketika Berpindah ke Halaman 2	80
11.8.	Membuat State Halaman.....	81
11.9.	Mengambil Data untuk Halaman Aktif.....	82
11.10.	Membuat Tombol Pagination	82
11.11.	Membuat Nomor Halaman Secara Dinamis.....	83
11.12.	Menampilkan Tombol dengan .map().....	84
11.13.	Memberikan Style pada Halaman Aktif	85
11.14.	Tombol Previous dan Next	85
11.15.	Membuat Komponen Pagination.....	88
11.16.	Ringkasan.....	92
BAB 12	EDITING	94
12.1.	Pendahuluan.....	94
12.2.	Apa Itu Editing.....	95
12.3.	Menambahkan Tombol Edit	96
12.4.	Membuat State Untuk Data Yang Diedit	97
12.5.	Membuat Form Editing	98
12.6.	Menggunakan State Pada Form Editing.....	100
12.7.	Menyimpan Perubahan Data	102
12.8.	Memperbaharui Data Map()	103
12.9.	Ringkasan.....	106
BAB 13	CONTEXT API	108

13.1.	Pendahuluan.....	108
13.2.	Mengenal Context API	109
13.3.	Membuat Context.....	111
13.4.	Membuat Provider	111
13.5.	Menggunakan Provider Pada CareerHub.....	113
13.6.	Menggunakan Context Dengan Usecontext().....	114
13.7.	Mengubah Data Pada Context.....	115
13.8.	Context API Pada CareeHub	116
13.9.	Context API Dan Props.....	117
13.10.	Ringkasan.....	118
BAB 14	REACT ROUTER.....	119
14.1.	Pendahuluan.....	119
14.2.	Apa Itu React Router	120
14.3.	Instalasi React Router	121
14.4.	Membuat Halaman CareerHub.....	122
14.5.	Membuat Route.....	123
14.6.	Menerapkan Routing Pada App.jsx	125
14.7.	Membuat Navigasi Antar Halaman.....	127
14.8.	Membuat Header CareerHub	128
14.9.	Menempatkan Header Pada Semua Halaman.....	129
14.10.	Link Vs <a>.....	130
14.11.	Menambahkan Halaman Detail Lowongan	131
14.12.	Navigasi Dari Job Card Ke Detail.....	132
14.13.	Mengambil Parameter URL	133
14.14.	Struktur Routing CareerHub.....	135

14.15.	Ringkasan.....	136
BAB 15 DEPLOYMENT		137
15.1.	Pendahuluan.....	137
15.2.	Apa Itu Vercel.....	137
15.3.	Mengapa Menggunakan Vercel.....	138
15.4.	Persiapan Sebelum Deployment	139
15.5.	Menyimpan Project Ke Github.....	140
15.6.	Membuat Akun Vercel.....	141
15.7.	Membuat Project Di Vercel.....	141
15.8.	Mengatur Project React	142
15.9.	Build Command Dan Output Directory.....	143
15.10.	Melakukan Deployment.....	144
15.11.	Production Dan Preview Deployment.....	145
15.12.	Deployment Menggunakan Vercel CLI	146
15.13.	Masalah Yang Sering Terjadi.....	146
15.14.	Ringkasan.....	147

BAB I

PENGENALAN JAVASCRIPT MODERN (ES6)

1.1. Pendahuluan

Sebelum mempelajari React, peserta perlu memahami terlebih dahulu JavaScript Modern atau yang sering disebut **ECMAScript 6 (ES6)**. React dikembangkan menggunakan sintaks JavaScript modern sehingga sebagian besar contoh kode, dokumentasi resmi, maupun proyek nyata menggunakan fitur-fitur ES6.

JavaScript Modern menghadirkan berbagai penyempurnaan dibandingkan JavaScript versi sebelumnya. Sintaks menjadi lebih ringkas, mudah dibaca, serta membantu pengembang menulis kode yang lebih terstruktur. Beberapa fitur yang paling sering digunakan dalam pengembangan React antara lain **let dan const**, **Arrow Function**, **Template Literals**, **Destructuring Assignment**, serta **Spread dan Rest Operator**.

Pada bab ini peserta akan mempelajari konsep dasar dari setiap fitur tersebut beserta contoh penerapannya sehingga lebih siap memasuki materi React pada pertemuan berikutnya.

1.2. Variabel (let dan const)

Variabel merupakan tempat untuk menyimpan suatu nilai di dalam program. Nilai tersebut dapat berupa angka, teks (string), boolean, array, object, maupun tipe data lainnya. Pada JavaScript versi lama, deklarasi variabel dilakukan menggunakan keyword **var**. Namun sejak hadirnya ES6, penggunaan **var** mulai ditinggalkan dan digantikan oleh **let** dan **const**.

Penggunaan **let** dan **const** membuat kode menjadi lebih aman karena memiliki aturan ruang lingkup (scope) yang lebih jelas dibandingkan **var**.

a) let

Keyword let digunakan ketika nilai suatu variabel dapat berubah selama program dijalankan.

Sintaks

```
let nama = "Andi";
```

Nilainya dapat diubah.

```
let nama = "Andi";
```

```
nama = "Budi";
```

```
console.log(nama); // Budi
```

Pada contoh di atas, variabel nama awalnya berisi nilai Andi, kemudian nilainya diubah menjadi Budi.

b) const

Keyword const digunakan untuk menyimpan nilai yang tidak akan diganti.

Sintaks

```
const kampus = "Universitas Nasional PASIM Bandung";
```

Jika nilainya diubah

```
const kampus = "Universitas Nasional PASIM Bandung";
```

```
kampus = "Universitas Nasional PASIM Jakarta"; // error
```

akan menghasilkan error karena variabel yang dideklarasikan menggunakan const tidak dapat diisi ulang.

Perbedaan let dan const

let	const
Nilai dapat diubah	Nilai tidak dapat diubah
Digunakan jika data berubah	Digunakan jika data tetap

Sering digunakan untuk counter atau looping	Paling sering digunakan di react
---	----------------------------------

Dalam pengembangan React, hampir semua variabel menggunakan const, sedangkan let hanya digunakan apabila nilainya memang berubah.

Contoh Kasus

Misalkan kita ingin menyimpan informasi seorang mahasiswa.

```
const nama = "Budi";
const jurusan = "Teknik Informatika";
let semester = 4;

semester = 5;

console.log(nama); // Budi
console.log(jurusan); // Teknik Informatika
console.log(semester); // 5
```

Pada contoh tersebut, nama dan jurusan tidak berubah sehingga menggunakan const, sedangkan semester berubah sehingga menggunakan let.

1.3. Arrow Function

Function merupakan sekumpulan kode yang dibuat untuk menjalankan suatu tugas tertentu. Dengan menggunakan function, kita tidak perlu menulis kode yang sama berulang kali.

Sebelum ES6, function ditulis menggunakan keyword function.

Contoh:

```
function sapa() {
  console.log("Halo");
}
```

Pada ES6 diperkenalkan Arrow Function, yaitu cara penulisan function yang lebih singkat dan lebih mudah dibaca.

Sintaks Arrow Function

Penulisan biasa

```
function tambah (a, b) {  
  return a + b;  
}
```

Ditulis menjadi

```
const tambah = (a, b) => {  
  return a + b;  
}
```

Apabila hanya memiliki satu baris kode, penulisannya dapat dipersingkat.

```
const tambah = (a, b) => a + b;
```

Mengapa React Menggunakan Arrow Function?

Pada React, hampir seluruh komponen modern ditulis menggunakan Arrow Function karena:

- Penulisannya lebih singkat.
- Lebih mudah dibaca.
- Tidak memiliki binding this seperti function biasa.
- Menjadi standar pada Functional Component.

Contoh sederhana komponen React

```
const home = () => {  
  return h1("Selamat Datang di CareerHub");  
}
```

Contoh Kasus

Membuat fungsi menghitung luas persegi panjang.

```
const hitungLuas = (panjang, lebar) => {
  return panjang * lebar;
}

console.log(hitungLuas(10, 5)); // 50
```

Pada contoh tersebut, function menerima dua parameter yaitu panjang dan lebar, kemudian mengembalikan hasil perkalian keduanya.

1.4. Template Literals

Template Literal merupakan cara baru dalam membuat string pada JavaScript menggunakan tanda backtick (`). Dengan Template Literal, penggabungan teks dan variabel menjadi lebih sederhana dibandingkan menggunakan operator (+).

Cara Lama

```
const nama = "Andi";
console.log("Halo, " + nama + "!"); // HaLo, Andi!
```

Menggunakan Template Literal

```
const nama = "Andi";
console.log(`Halo, ${nama}!`); // HaLo, Andi!
```

Keunggulan Template Literal

- Penulisan lebih rapi.
- Tidak perlu menggunakan tanda (+).
- Mudah membaca string yang panjang.
- Sangat sering digunakan pada React untuk menampilkan data secara dinamis.

Contoh Kasus

```
const produk = "Laptop";
const harga = 15000000;
console.log(`Produk: ${produk}, Harga: Rp. ${harga.toLocaleString("id-ID")}`);
// Produk: Laptop, Harga: Rp. 15.000.000
```

1.5. Destructuring Assignment

Destructuring Assignment merupakan fitur ES6 yang digunakan untuk mengambil nilai dari object atau array secara langsung ke dalam variabel.

Tanpa Destructuring, kita harus mengambil setiap properti satu per satu.

Misalnya terdapat object berikut.

```
const mahasiswa = {  
  nama: "Andi",  
  jurusan: "Teknik Informatika",  
  semester: 4  
};
```

Cara biasa

```
const nama = mahasiswa.nama;  
const jurusan = mahasiswa.jurusan;  
const semester = mahasiswa.semester;
```

Cara ES6

```
const { nama, jurusan, semester } = mahasiswa;
```

Destructuring Array

```
const warna = ["Merah", "Hijau", "Biru"];  
const [warna1, warna2, warna3] = warna;
```

Mengapa Penting di React?

React menggunakan destructuring hampir di setiap component.

Contoh

```
const profile = ({ nama, jurusan, semester }) => {  
  return (  
    <div>  
      <h1>{nama}</h1>  
      <p>{jurusan}</p>  
      <p>{semester}</p>  
    </div>  
  )  
}
```

Tanpa destructuring, kita harus menulis:

```
// props.nama  
// props.jurusan  
// props.semester
```

Contoh Kasus

```
const mahasiswa = {  
  nama = "Wahyu",  
  pekerjaan = "Middleware Engineer",  
  kota = "Bandung"  
}  
  
const {nama, pekerjaan, kota} = user;  
  
console.log(nama);  
console.log(pekerjaan);  
console.log(kota);
```

1.6. Spread dan Rest Operator

Spread (...) dan Rest (...) menggunakan simbol yang sama, tetapi memiliki fungsi yang berbeda.

- Spread Operator digunakan untuk menguraikan isi array atau object.
- Rest Operator digunakan untuk mengumpulkan beberapa nilai menjadi satu.

Spread Operator

Spread digunakan untuk menyalin atau menggabungkan data.

Contoh array

```
const angka1 = [1, 2, 3];  
const angka2 = [...angka1];  
  
console.log(angka2); //Output: [1, 2, 3]
```

Menggabungkan array

```
const frontEnd = ["HTML", "CSS", "JavaScript"];  
const backEnd = ["Node.js", "Express", "MongoDB"];  
const fullStack = [...frontEnd, ...backEnd];
```

```
console.log(fullStack); // Output: ["HTML", "CSS", "JavaScript", "Node.js",  
"Express", "MongoDB"]
```

Menggabungkan object

```
const mahasiswa1 = {  
  nama: "Andi",  
  umur: 20  
};  
const mahasiswa2 = {  
  jurusan: "Teknik Informatika",  
  kampus: "Universitas Nasional PASIM"  
};  
const dataMahasiswa = { ...mahasiswa1, ...mahasiswa2 };  
console.log(dataMahasiswa); // Output: { nama: "Andi", umur: 20, jurusan:  
"Teknik Informatika", kampus: "Universitas Nasional PASIM" }
```

Rest Operator

Rest digunakan ketika jumlah parameter belum diketahui.

```
// rest operator  
const jumlahKan = (...angka) => {  
  let total = 0;  
  for (let nilai of angka) {  
    total += nilai;  
  }  
  return total;  
}  
console.log(jumlahKan(1, 2, 3, 4, 5)); // Output: 15
```

Pada contoh tersebut, semua parameter akan dikumpulkan menjadi sebuah array bernama angka.

1.7. Ringkasan Bab

Pada bab ini telah dipelajari berbagai fitur penting JavaScript Modern yang menjadi dasar dalam pengembangan aplikasi React. Peserta memahami penggunaan **let** dan **const** untuk mendeklarasikan variabel, **Arrow Function** untuk menulis fungsi yang lebih ringkas, **Template Literals** untuk membangun string dinamis, **Destructuring Assignment** untuk mengambil data dari object atau array dengan lebih mudah, serta **Spread** dan **Rest Operator** untuk mengelola data secara efisien.

Seluruh konsep tersebut akan terus digunakan pada bab-bab berikutnya saat mulai membangun komponen React, mengelola *props*, *state*, dan melakukan manipulasi data dalam aplikasi. Dengan menguasai dasar-dasar ES6 ini, peserta akan lebih mudah memahami sintaks dan pola pengembangan React modern.

BAB 2

PENGENALAN NODE.JS, NPM DAN PNPM

2.1. Pendahuluan

Sebelum mulai membangun aplikasi menggunakan React, peserta perlu mempersiapkan lingkungan pengembangan (*development environment*) yang akan digunakan selama pelatihan. React tidak dapat dijalankan hanya menggunakan browser seperti HTML, CSS, dan JavaScript biasa. Dibutuhkan beberapa perangkat pendukung, seperti **Node.js** serta **Package Manager** yang berfungsi untuk mengelola berbagai library dan dependency yang digunakan dalam sebuah proyek.

Pada bab ini peserta akan mempelajari cara menginstal **Node.js**, mengenal npm dan pnpm sebagai package manager, serta memahami bagaimana membuat dan mengelola sebuah proyek JavaScript menggunakan kedua package manager tersebut. Materi ini menjadi dasar sebelum memasuki proses pembuatan aplikasi React pada bab berikutnya.

2.2. Node.js

Node.js merupakan sebuah **runtime environment** yang memungkinkan JavaScript dijalankan di luar browser. Sebelum Node.js diperkenalkan, JavaScript hanya dapat dijalankan pada browser seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge. Dengan Node.js, JavaScript dapat digunakan untuk membangun aplikasi server, menjalankan script, hingga mengembangkan aplikasi React.

React sendiri memanfaatkan Node.js sebagai lingkungan untuk menjalankan berbagai tools pengembangan, seperti Vite, npm, dan package lainnya.

Catatan: Node.js bukanlah framework maupun bahasa pemrograman baru, melainkan sebuah runtime yang dibangun menggunakan mesin JavaScript **V8 Engine** milik Google Chrome.

Fungsi Node.js

Beberapa fungsi utama Node.js antara lain:

- Menjalankan JavaScript di luar browser.
- Menyediakan package manager (npm).
- Menjalankan development server React.
- Mengelola dependency proyek.
- Menjalankan berbagai tools pengembangan.

Instalasi Node.js

1. Langkah-langkah instalasi Node.js:
2. Buka website resmi Node.js.
3. Unduh versi LTS (Long Term Support).
4. Jalankan installer.
5. Klik Next hingga proses instalasi selesai.
6. Restart terminal atau Command Prompt.

Setelah selesai, lakukan pengecekan instalasi.

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The prompt '1' is followed by the command 'node -v'.

Contoh output

A dark-themed terminal window with three colored window control buttons in the top-left corner. The prompt '1' is followed by the output 'v24.17.0'.

Kemudian cek npm

A dark-themed terminal window with three colored window control buttons in the top-left corner. The prompt '1' is followed by the command 'npm -v'.

Contoh output



```
1 11.13.0
```

Jika kedua perintah tersebut menampilkan nomor versi, maka instalasi berhasil.

2.3. Pengenalan npm (Node Package Manager)

Apa itu npm?

npm (Node Package Manager) merupakan package manager bawaan yang otomatis terpasang ketika menginstal Node.js.

Package manager berfungsi untuk:

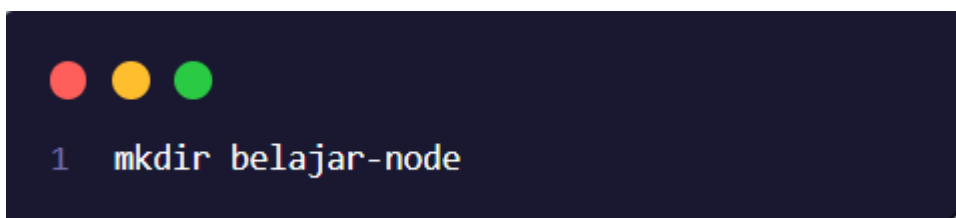
- menginstal library
- memperbarui package
- menghapus package
- menjalankan script proyek

Saat membuat aplikasi React, npm akan digunakan untuk mengunduh berbagai dependency yang dibutuhkan.

Inisialisasi Project

Misalkan kita ingin membuat project baru.

Buat folder terlebih dahulu.



```
1 mkdir belajar-node
```

Masuk ke folder.

```
1 cd belajar-node
```

Kemudian jalankan

```
1 npm init
```

Atau

```
1 npm init -y
```

Perintah tersebut akan menghasilkan file **package.json**.

Apa itu package.json?

package.json merupakan file konfigurasi utama dalam sebuah project Node.js.

Contoh sederhana

```
{
  "name": "belajar-node",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
}
```

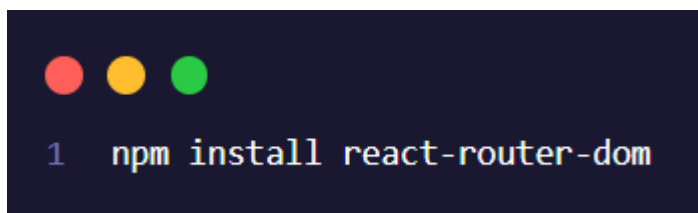
```
"dependencies": {  
  "react-icon": "^1.0.0",  
  "react-router-dom": "^7.18.1"  
}
```

File ini menyimpan informasi seperti:

- nama project
- versi project
- dependency
- script yang dapat dijalankan

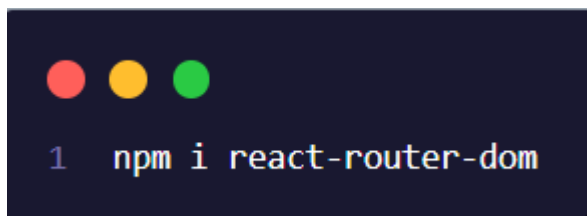
Menginstal Package

Sebagai contoh kita akan menginstal package react-router-dom.



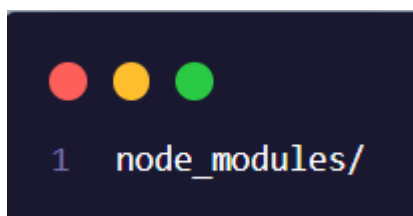
```
1 npm install react-router-dom
```

Atau



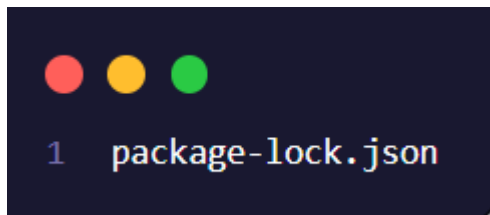
```
1 npm i react-router-dom
```

Setelah berhasil akan muncul folder



```
1 node_modules/
```

Serta file

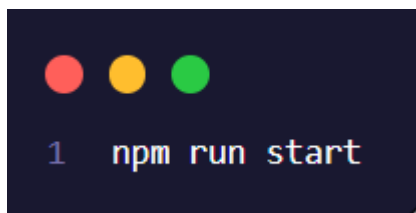


Menjalankan Script

Misalkan terdapat script

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "node index.js"  
},
```

Maka dapat dijalankan dengan menggunakan



2.4. Pengenalan pnpm

Apa itu pnpm?

Selain npm, terdapat package manager lain yang cukup populer yaitu pnpm (Performant Node Package Manager).

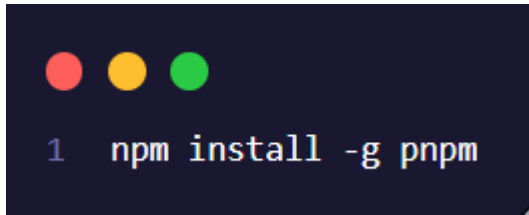
pnpm memiliki fungsi yang sama dengan npm, namun memiliki beberapa keunggulan seperti:

- proses instalasi lebih cepat
- penggunaan ruang penyimpanan lebih hemat
- dependency tidak banyak duplikasi
- performa lebih baik untuk project besar

Karena alasan tersebut, pada pelatihan ini kita akan menggunakan pnpm sebagai package manager utama.

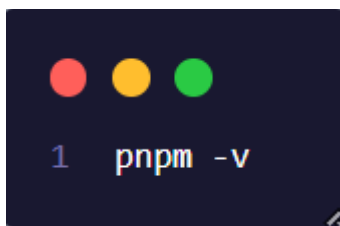
Instalasi pnpm

Jalankan perintah berikut.



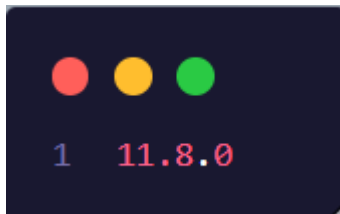
```
1 npm install -g pnpm
```

Setelah selesai, cek versinya



```
1 pnpm -v
```

Contoh output



```
1 11.8.0
```

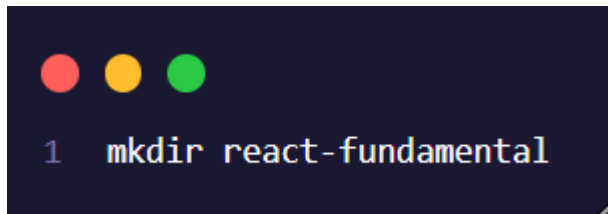
Perbedaan npm dan pnpm

npm	pnpm
Bawaan node.js	Perlu instalasi
Instalasi relatif lebih lambat	Instalasi lebih cepat
Menggunakan storage lebih besar	Lebih hemat storage
Cocok untuk project kecil	Cocok untuk project besar

2.5. Membuat dan Mengelola Project

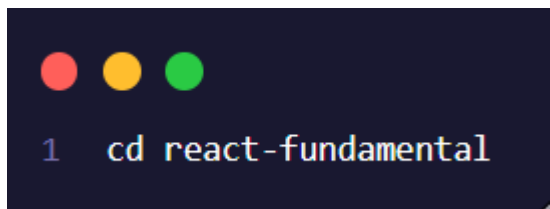
Setelah memiliki Node.js dan pnpm, kita dapat mulai membuat project

Buat folder baru

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 mkdir react-fundamental` is entered and executed.

```
1 mkdir react-fundamental
```

Masuk ke folder

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 cd react-fundamental` is entered and executed.

```
1 cd react-fundamental
```

Inisialisasi project menggunakan pnpm

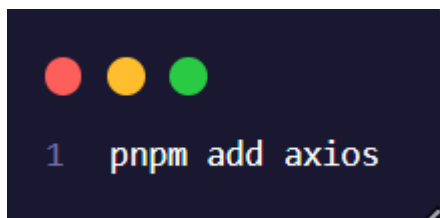
A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 pnpm init` is entered and executed.

```
1 pnpm init
```

Akan muncul file **package.json**.

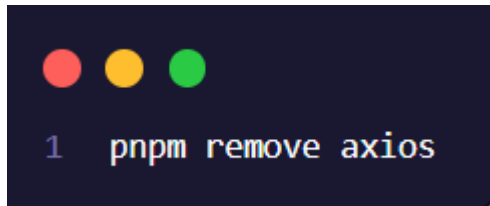
Menginstal Package

Misalkan ingin menginstal package Axios.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 pnpm add axios` is entered and executed.

```
1 pnpm add axios
```

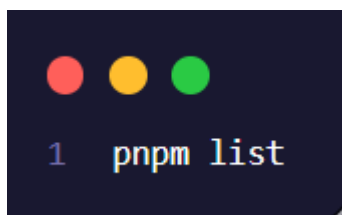
Menghapus Package

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 pnpm remove axios` is entered in a light blue monospace font.

Memperbarui Package

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 pnpm update` is entered in a light blue monospace font.

Melihat Package yang Terinstal

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command `1 pnpm list` is entered in a light blue monospace font.

2.6. Ringkasan Bab

Pada bab ini peserta telah mempelajari dasar-dasar lingkungan pengembangan JavaScript modern yang akan digunakan selama pelatihan React. Peserta memahami fungsi **Node.js** sebagai runtime JavaScript, mengenal npm dan pnpm sebagai package manager, serta mampu melakukan instalasi, membuat proyek baru, dan mengelola dependency menggunakan berbagai perintah dasar.

Pemahaman terhadap materi pada bab ini sangat penting karena seluruh proses pengembangan aplikasi React pada bab-bab selanjutnya akan memanfaatkan **Node.js** dan **pnpm** sebagai alat utama dalam menjalankan proyek, menginstal library, dan mengelola kebutuhan aplikasi. Dengan lingkungan pengembangan yang telah disiapkan, peserta siap melanjutkan ke bab berikutnya untuk mulai membuat proyek React menggunakan **Vite**.

BAB 3

MENGGUNAKAN PNPM DAN VITE

3.1. Pendahuluan

Pada bab sebelumnya, peserta telah mempelajari cara menginstal Node.js, mengenal npm dan pnpm, serta memahami fungsi package manager dalam pengembangan aplikasi JavaScript. Setelah lingkungan pengembangan siap digunakan, langkah berikutnya adalah membuat proyek React yang akan digunakan selama proses pelatihan.

Dalam pelatihan ini, peserta akan mengembangkan sebuah aplikasi sederhana bernama CareerHub, yaitu sebuah aplikasi Mini Job Portal yang digunakan untuk menampilkan daftar lowongan pekerjaan. Aplikasi ini akan terus dikembangkan secara bertahap hingga pertemuan terakhir, sehingga setiap materi yang dipelajari akan langsung diterapkan pada proyek yang sama.

Pada bab ini peserta akan mempelajari cara membuat proyek React menggunakan Vite, menjalankan aplikasi pertama, memahami struktur folder proyek, serta mengenal beberapa perintah dasar pnpm yang digunakan dalam proses pengembangan.

Studi Kasus

Project Pelatihan : CareerHub

Selama pelatihan React Fundamental, seluruh materi akan diterapkan pada satu proyek yang sama, yaitu CareerHub.

CareerHub merupakan aplikasi sederhana yang berfungsi untuk menampilkan informasi lowongan pekerjaan. Seiring bertambahnya materi, aplikasi ini akan dikembangkan menjadi lebih lengkap dengan berbagai fitur seperti pencarian, pengurutan data, pagination, editing data, routing, hingga deployment ke internet.

Pada bab ini peserta belum akan membuat fitur-fitur tersebut. Fokus utama adalah membuat proyek React dan memastikan aplikasi dapat dijalankan dengan baik.

3.2. Mengetahui Vite

Vite merupakan sebuah build tool modern yang digunakan untuk membuat dan menjalankan aplikasi JavaScript seperti React, Vue, maupun framework lainnya.

Vite dirancang agar proses pengembangan aplikasi menjadi lebih cepat dibandingkan build tool sebelumnya, seperti Create React App (CRA). Dengan Vite, proses menjalankan aplikasi dan memperbarui perubahan kode (Hot Module Replacement) berlangsung hampir secara instan sehingga sangat membantu selama proses pengembangan.

Pada pelatihan ini seluruh proyek React akan dibuat menggunakan Vite.

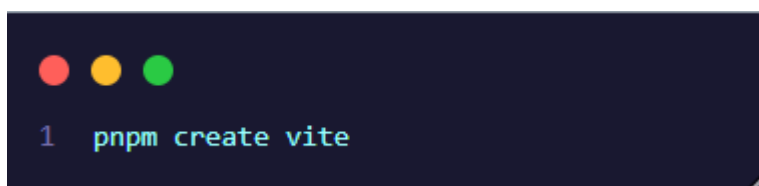
Keunggulan Vite

Beberapa kelebihan Vite antara lain:

- Proses pembuatan proyek lebih cepat.
- Startup development server sangat ringan.
- Mendukung Hot Module Replacement (HMR).
- Konfigurasi sederhana.
- Cocok digunakan untuk pengembangan React modern.

3.3. Membuat Project

Buka Terminal atau Command Prompt kemudian jalankan perintah berikut.

A screenshot of a terminal window with a dark background. At the top left, there are three colored circles: red, yellow, and green. Below them, the text '1 pnpm create vite' is displayed in a light blue font. The terminal window has a small icon in the bottom right corner.

Selanjutnya akan muncul beberapa pertanyaan.

```
◇ Project name:
  careerhub
◇ Select a framework:
  React
◇ Select a variant:
  JavaScript
◇ Which linter to use?
  ESLint
◇ Install with pnpm and start now?
  Yes
◇ Scaffolding project in D:\PELATIHAN\pelatihan-react-fundamental-2026\pelatihan\pertemuan-3\careerhub...
◇ Installing dependencies with pnpm...

Update available! 11.8.0 → 11.10.0.
Changelog: https://pnpm.io/v/11.10.0
To update, run: pnpm self-update

Downloading @rolldown/binding-win32-x64-msvc@1.1.5: 7.80 MB/7.80 MB, done
Packages: +135
Progress: resolved 166, reused 128, downloaded 13, added 135, done
```

Apabila proses berhasil maka akan terbentuk folder project baru bernama careerhub.

Masuk ke Folder Project

Masuk ke folder project menggunakan perintah berikut.

```
1 cd careerhub
```

Menjalankan Project

Setelah dependency selesai diinstal, jalankan project.

```
1 pnpm dev
```

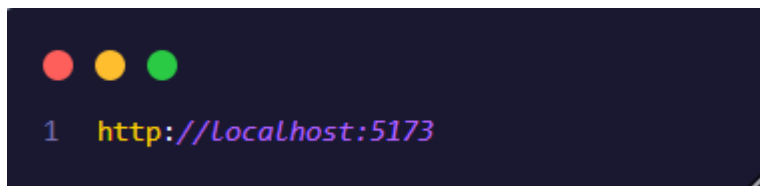
Apabila berhasil akan muncul tampilan seperti berikut.

```
PS D:\PELATIHAN\pelatihan-react-fundamental-2026\pelatihan\pertemuan-3\careerhub> pnpm dev
$ vite

VITE v8.1.3 ready in 262 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

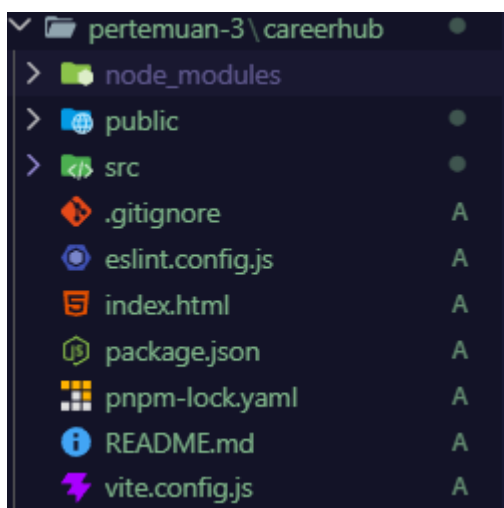
Buka browser kemudian akses alamat tersebut.



Jika berhasil maka akan muncul halaman awal React bawaan Vite.

3.4.Struktur Folder Project

Setelah project berhasil dibuat, akan muncul struktur folder seperti berikut.



Berikut penjelasan fungsi masing-masing folder.

Folder	Fungsi
src	Tempat seluruh kode React ditulis.

public	Menyimpan file statis seperti gambar atau favicon.
node_modules	Menyimpan seluruh dependency project.
package.json	Menyimpan konfigurasi project serta daftar dependency.
pnpm-lock.yaml	Menyimpan versi package yang digunakan.
vite.config.js	File konfigurasi Vite.

3.5. Mengenal File package.json

Setiap project React memiliki sebuah file bernama package.json.

File ini berisi informasi mengenai project, dependency yang digunakan, serta script yang dapat dijalankan.

Contoh isi package.json.

```
{
  "name": "belajar-node",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "react-icon": "^1.0.0",
    "react-router-dom": "^7.18.1"
  }
}
```

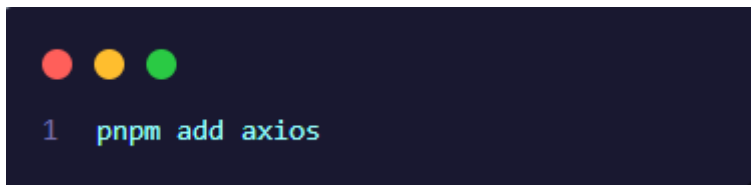
Script yang paling sering digunakan adalah:

Script	Fungsi
pnpm dev	Menjalankan aplikasi.
pnpm build	Melakukan build project.
pnpm preview	Menjalankan hasil build.

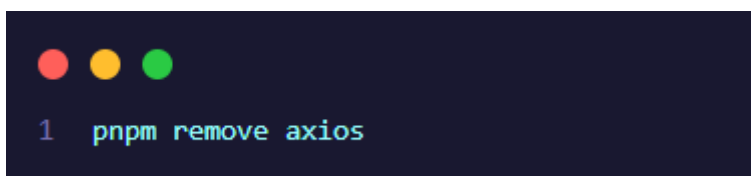
3.6. Perintah Dasar pnpm

Selain digunakan untuk membuat project, pnpm juga digunakan untuk mengelola dependency.

Menginstal Package

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command `1 pnpm add axios` is entered in a light blue monospace font.

Menghapus Package

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command `1 pnpm remove axios` is entered in a light blue monospace font.

Memperbarui Package

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command `1 pnpm update` is entered in a light blue monospace font.

Melihat Package yang Terpasang

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The text '1 pnpm list' is displayed in a light blue monospace font.

3.7. Ringkasan Bab

Pada bab ini peserta telah berhasil membuat proyek React menggunakan Vite dengan bantuan pnpm sebagai package manager. Peserta juga telah memahami struktur dasar proyek React, fungsi file package.json, serta beberapa perintah dasar pnpm yang sering digunakan dalam pengembangan aplikasi.

Project CareerHub yang telah dibuat akan menjadi studi kasus utama selama pelatihan. Pada bab-bab berikutnya, aplikasi ini akan dikembangkan secara bertahap hingga menjadi sebuah Mini Job Portal yang memiliki berbagai fitur seperti pencarian, pengurutan data, pagination, editing, routing, penggunaan Context API, dan deployment ke Vercel.

BAB 4

DASAR-DASAR REACT DAN KOMPONEN

4.1. Pendahuluan

Pada bab sebelumnya, peserta telah berhasil membuat project React menggunakan Vite serta menjalankan aplikasi pertama. Namun, tampilan yang muncul masih merupakan halaman bawaan dari Vite dan belum mencerminkan aplikasi yang akan dikembangkan.

Pada bab ini peserta akan mulai membangun aplikasi CareerHub menggunakan React. Materi yang dipelajari meliputi pengenalan React, konsep JSX, pembuatan Functional Component, serta cara menggunakan komponen di dalam aplikasi. Dengan memahami materi pada bab ini, peserta akan mampu membangun tampilan aplikasi yang tersusun dari beberapa komponen yang saling terhubung.

4.2. Apa itu React?

React merupakan library JavaScript yang digunakan untuk membangun User Interface (UI) atau antarmuka pengguna.

React dikembangkan oleh Meta (Facebook) dan menggunakan konsep Component-Based Architecture, yaitu sebuah aplikasi dibangun dari kumpulan komponen-komponen kecil yang dapat digunakan kembali (reusable).

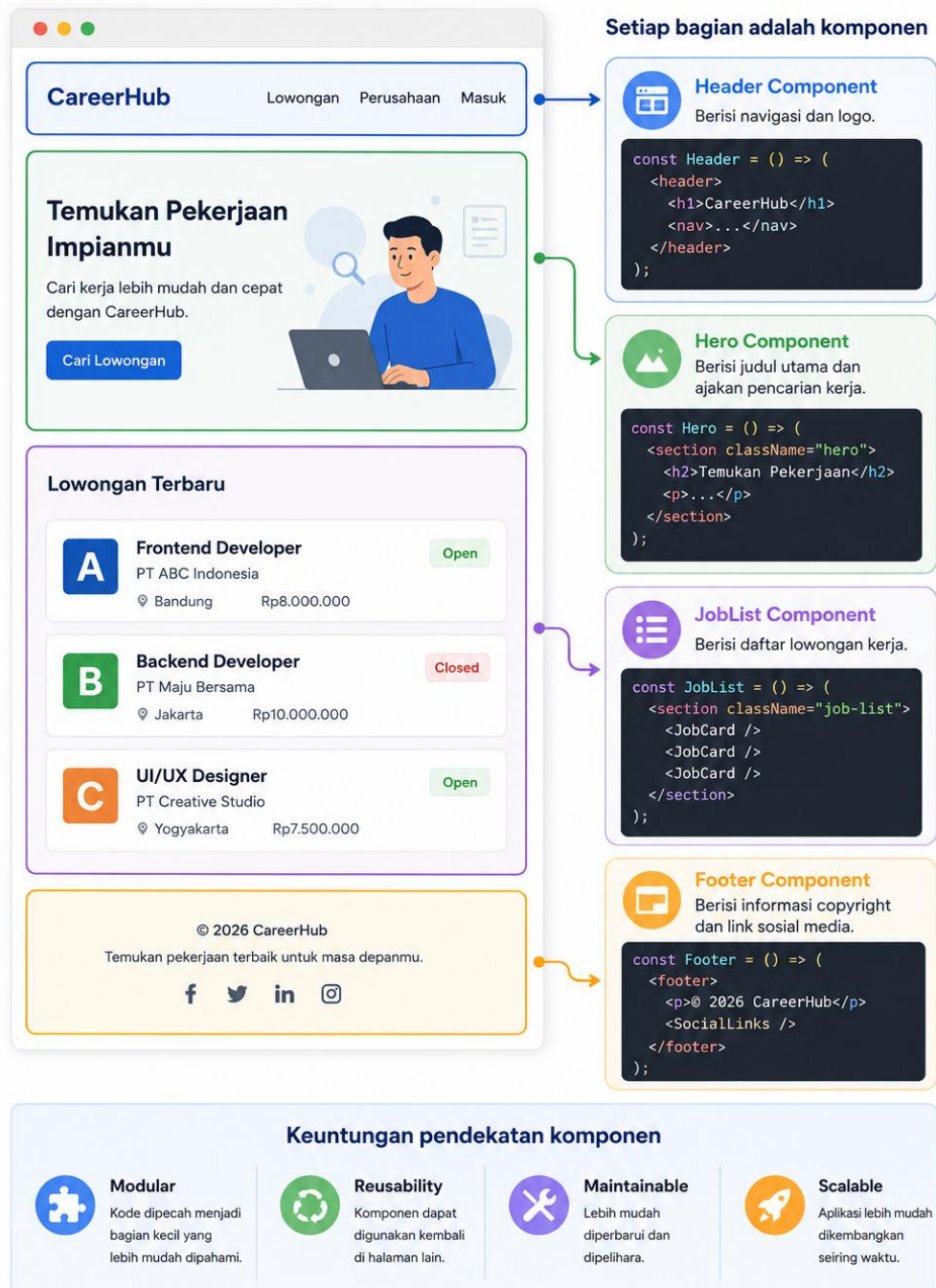
Sebagai contoh, sebuah website biasanya terdiri dari beberapa bagian seperti:

- Header
- Navigation Bar
- Hero Section
- Content
- Footer

Pada React, setiap bagian tersebut dapat dibuat menjadi komponen yang berdiri sendiri.

CareerHub

Aplikasi dibangun dari komponen-komponen yang berdiri sendiri



4.3.Mengenal JSX

JSX (JavaScript XML) merupakan sintaks yang memungkinkan kita menulis kode HTML di dalam JavaScript.

Meskipun terlihat seperti HTML, JSX sebenarnya akan diubah menjadi kode JavaScript oleh React.

Contoh JSX:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is as follows:

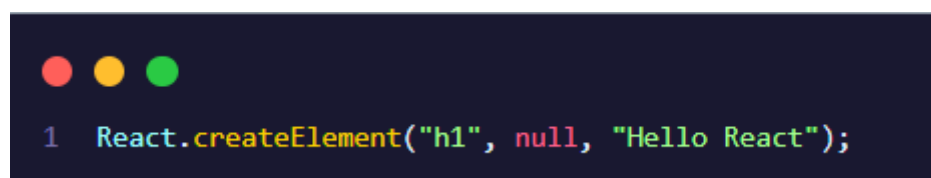
```
1 function App() {  
2   return (  
3     <h1>Hello React</h1>  
4   );  
5 }
```

Mengapa Menggunakan JSX?

- JSX memiliki beberapa kelebihan, di antaranya:
- Penulisan kode lebih mudah dibaca.
- Tampilan dan logika dapat ditulis dalam satu file.
- Mempermudah pembuatan komponen.

Sebagai contoh:

Tanpa JSX:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is as follows:

```
1 React.createElement("h1", null, "Hello React");
```

Dengan JSX:

```
1 <h1>Hello React</h1>
```

Tentunya penulisan menggunakan JSX lebih sederhana dan mudah dipahami.

Aturan Penulisan JSX

Dalam menggunakan JSX terdapat beberapa aturan yang perlu diperhatikan.

1. Hanya memiliki satu elemen utama

Benar:

```
1 return (  
2   <div>  
3     <h1>CareerHub</h1>  
4     <p>Portal Lowongan Kerja</p>  
5   </div>  
6 );
```

Salah:

```
1 return (  
2   <h1>CareerHub</h1>  
3   <p>Portal Lowongan Kerja</p>  
4 );
```

2. Gunakan className

Karena class merupakan keyword JavaScript, maka React menggunakan className.

```
1 <div className="container">  
2   Hello React  
3 </div>
```

3. Gunakan Kurung Kurawal {}

4.4. Mengetahui Komponen

Pengertian Komponen

Komponen merupakan bagian kecil dari sebuah tampilan yang dapat digunakan berulang kali.

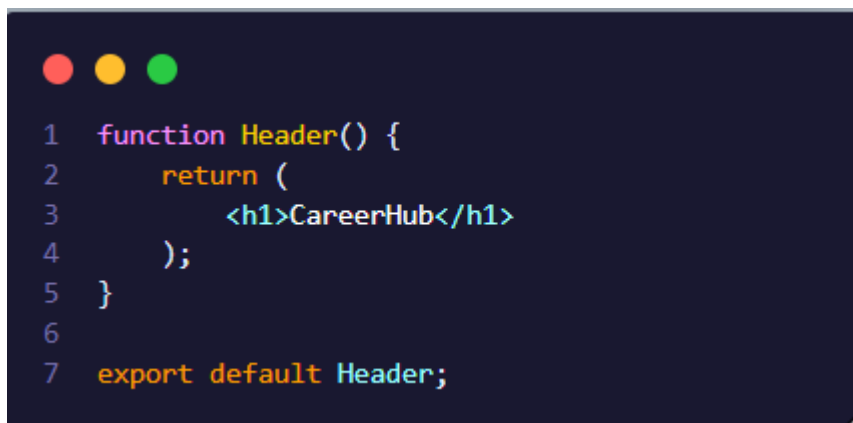
Sebagai contoh, pada aplikasi CareerHub nantinya terdapat beberapa bagian seperti:

- Header
- Hero
- Job Card
- Footer

Keempat bagian tersebut akan dibuat menjadi komponen yang terpisah.

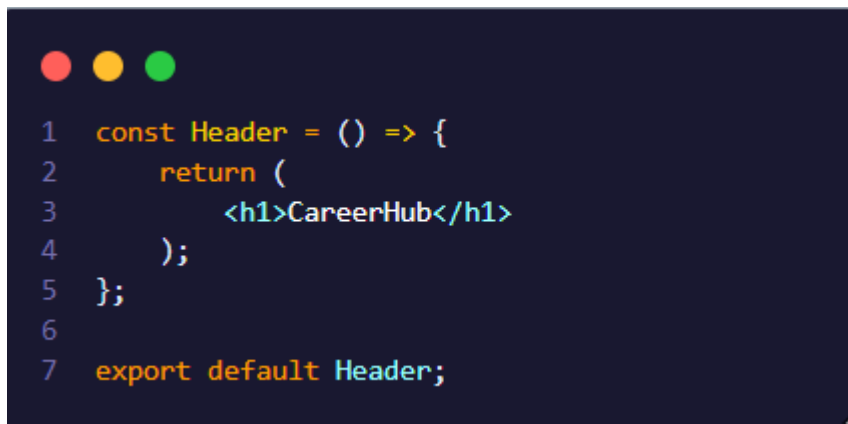
Functional Component

Pada React modern, komponen dibuat menggunakan Function.



```
1 function Header() {  
2   return (  
3     <h1>CareerHub</h1>  
4   );  
5 }  
6  
7 export default Header;
```

Selain menggunakan function biasa, komponen juga dapat dibuat menggunakan Arrow Function.



```

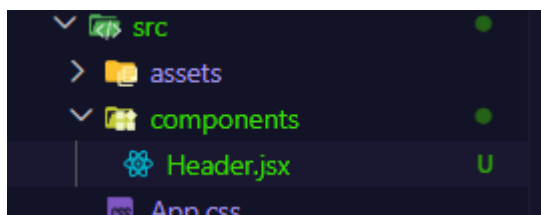
1  const Header = () => {
2      return (
3          <h1>CareerHub</h1>
4      );
5  };
6
7  export default Header;

```

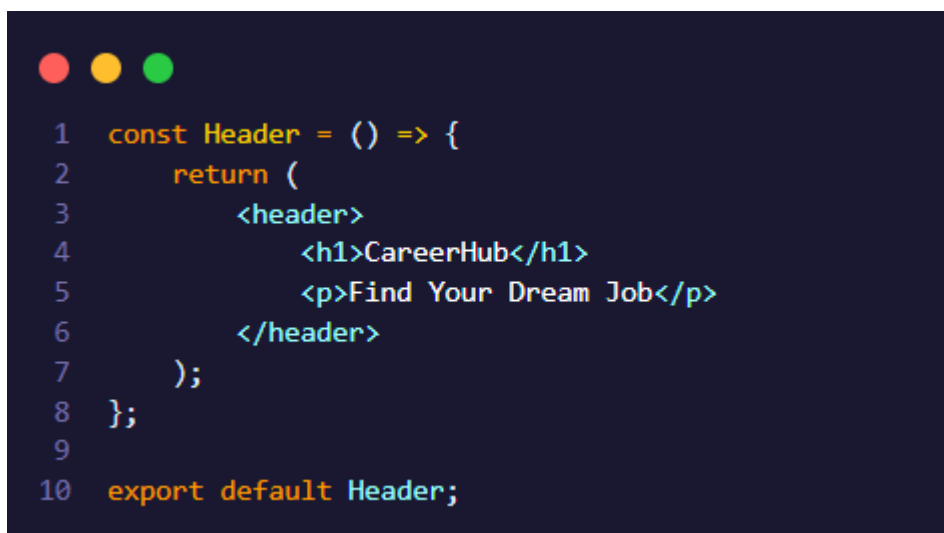
Cara kedua merupakan cara yang paling sering digunakan pada React modern.

4.5.Membuat Komponen Pertama

Pada folder src, buat folder baru bernama components, kemudian buat file Header.jsx



Isi file tersebut:



```

1  const Header = () => {
2      return (
3          <header>
4              <h1>CareerHub</h1>
5              <p>Find Your Dream Job</p>
6          </header>
7      );
8  };
9
10 export default Header;

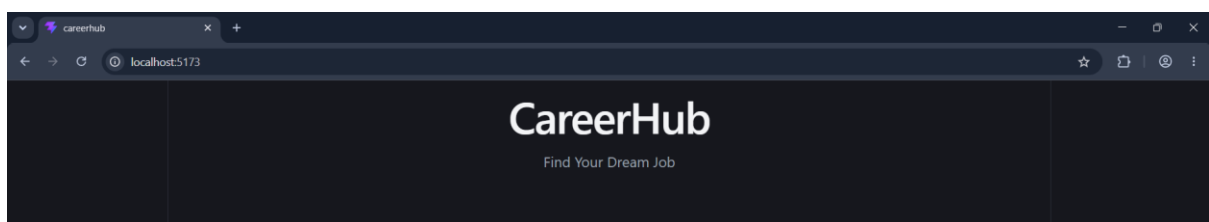
```

4.6. Menggunakan Komponen

Setelah komponen dibuat, langkah berikutnya adalah menggunakannya pada App.jsx.

```
1  import Header from "../components/Header";
2
3  function App() {
4    return (
5      <>
6        <Header />
7      </>
8    );
9  }
10
11 export default App;
```

Ketika aplikasi dijalankan, maka akan muncul:



4.7. Ringkasan Bab

Pada bab ini peserta telah mempelajari konsep dasar React, mengenal JSX sebagai sintaks untuk membangun antarmuka pengguna, serta memahami bagaimana aplikasi React disusun menggunakan Functional Component. Peserta juga telah berhasil membuat beberapa komponen sederhana dan menyusunnya menjadi tampilan awal aplikasi CareerHub.

Struktur komponen yang dibuat pada bab ini akan menjadi fondasi pengembangan aplikasi pada bab-bab selanjutnya. Pada Bab 5, peserta akan mempelajari Props, sehingga komponen JobCard tidak lagi menampilkan data statis, melainkan menerima data secara dinamis dari komponen induknya. Dengan

demikian, aplikasi CareerHub akan mulai menampilkan daftar lowongan pekerjaan yang lebih fleksibel dan mudah dikelola.

BAB 5

MENERUSKAN PROPS DAN STYLING KOMPONEN

5.1. Pendahuluan

Pada bab sebelumnya peserta telah mempelajari cara membuat komponen pada React. Namun setiap komponen masih menampilkan data yang bersifat statis (hardcode). Pendekatan tersebut kurang fleksibel karena setiap perubahan data harus dilakukan secara langsung di dalam komponen.

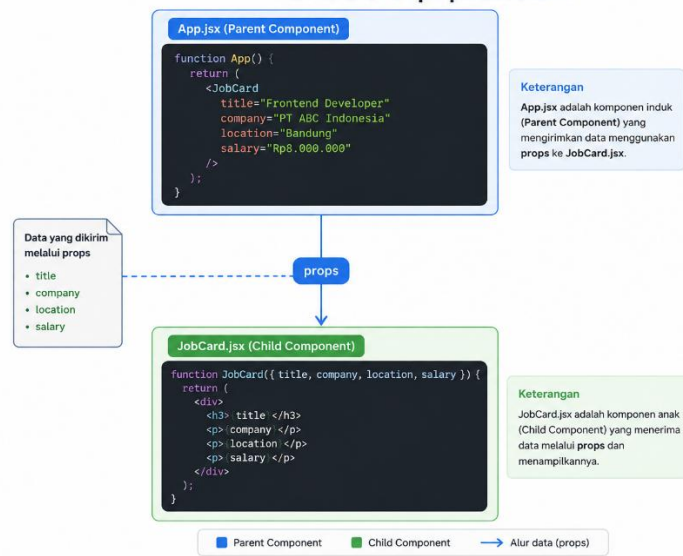
React menyediakan mekanisme Props (Properties) untuk mengirimkan data dari satu komponen ke komponen lainnya. Dengan menggunakan props, sebuah komponen dapat digunakan kembali dengan data yang berbeda tanpa perlu membuat komponen baru.

Selain mempelajari props, pada bab ini peserta juga akan mempelajari cara mempercantik tampilan aplikasi menggunakan Tailwind CSS, yaitu framework CSS yang banyak digunakan dalam pengembangan aplikasi React modern.

5.2. Apa itu Props?

Props (Properties) merupakan mekanisme yang digunakan React untuk mengirimkan data dari komponen induk (Parent Component) ke komponen anak (Child Component).

Ilustrasi Props pada React



Pada aplikasi CareerHub, data seperti:

- Judul pekerjaan
- Nama perusahaan
- Lokasi
- Gaji

tidak lagi ditulis langsung di dalam JobCard.jsx, melainkan dikirim melalui props.

5.3. Mengirim Props

Misalkan terdapat komponen berikut.

```
1 <JobCard  
2   title="Frontend Developer"  
3   company="PT ABC Indonesia"  
4   location="Bandung"  
5   salary="Rp8.000.000"  
6 />
```

Komponen JobCard sekarang menerima empat buah props.

5.4.Menerima Props

Props dapat diterima dengan dua cara.

Cara pertama

```
1  const JobCard = (props) => {  
2    return (  
3      <>  
4        <h3>{props.title}</h3>  
5      </>  
6    );  
7  };
```

Cara kedua (lebih disarankan)

```
1  const JobCard = ({ title, company, location, salary }) =>  
2  {  
3    return (  
4      <>  
5        <h3>{title}</h3>  
6      </>  
7    );  
8  };
```

Cara kedua menggunakan destructuring sehingga kode menjadi lebih ringkas.

Contoh lengkap

5.5.Styling React Menggunakan Tailwind CSS

Apa itu Tailwind CSS?

Tailwind CSS merupakan sebuah framework CSS berbasis Utility First yang menyediakan berbagai class siap pakai untuk membangun tampilan aplikasi tanpa harus menulis file CSS secara manual.

Berbeda dengan CSS konvensional yang mengharuskan kita membuat selector dan class sendiri, Tailwind CSS memungkinkan kita langsung memberikan styling pada elemen melalui atribut `className`.

Beberapa keuntungan menggunakan Tailwind CSS antara lain:

- Tidak perlu membuat file CSS untuk styling sederhana.
- Penulisan kode lebih cepat dan efisien.
- Mudah membuat tampilan yang responsif.
- Banyak digunakan pada proyek React modern.

Oleh karena itu, pada modul ini seluruh proses styling aplikasi CareerHub akan menggunakan Tailwind CSS.

5.6. Instalasi Tailwind CSS

Sebelum menggunakan Tailwind CSS, kita perlu melakukan proses instalasi pada project React yang telah dibuat pada bab sebelumnya.

Buka terminal pada folder project CareerHub, kemudian jalankan perintah berikut.

Install Tailwind CSS

```
\careerhub> npm add tailwindcss @tailwindcss/vite
```

Konfigurasi Vite

Buka file `vite.config.js`, kemudian ubah menjadi seperti berikut.

```

1  import { defineConfig } from "vite";
2  import react from "@vitejs/plugin-react";
3  import tailwindcss from "@tailwindcss/vite";
4
5  export default defineConfig({
6    plugins: [
7      react(),
8      tailwindcss(),
9    ],
10 });

```

Import Tailwind CSS

Buka file `src/index.css` (atau `src/main.css` sesuai struktur project), kemudian tambahkan baris berikut.

5.7. Mengenal Utility Class Tailwind CSS

Tailwind CSS menyediakan banyak utility class yang dapat digunakan untuk mengatur tampilan suatu komponen.

Berikut beberapa utility class yang paling sering digunakan selama pelatihan.

Utility	Fungsi	Contoh
<code>bg-*</code>	Warna latar belakang	<code>bg-blue-600</code>
<code>text-*</code>	Warna dan ukuran teks	<code>text-white</code> , <code>text-xl</code>
<code>font-*</code>	Ketebalan huruf	<code>font-bold</code>
<code>p-*</code>	Padding	<code>p-4</code>
<code>m-*</code>	Margin	<code>mt-5</code>
<code>rounded-*</code>	Sudut membulat	<code>rounded-xl</code>

Utility	Fungsi	Contoh
shadow-*	Bayangan	shadow-lg
border	Border	border
flex	Layout Flexbox	flex items-center
grid	Layout Grid	grid grid-cols-3

Contoh Penggunaan

```

1  const Header = () => {
2    return (
3      <header className="bg-blue-600 text-white p-6 shadow-md">
4        <h1 className="text-3xl font-bold">
5          CareerHub
6        </h1>
7
8        <p className="text-blue-100 mt-2">
9          Find Your Dream Job
10       </p>
11     </header>
12   );
13 };
14
15 export default Header;

```

BAB 6

CONDITIONAL RENDERING DAN LIST RENDERING

6.1. Pendahuluan

Pada bab sebelumnya, peserta telah mempelajari cara mengirimkan data menggunakan Props sehingga komponen JobCard dapat menampilkan informasi lowongan pekerjaan secara dinamis. Namun, setiap komponen JobCard masih harus ditulis satu per satu secara manual.

Pada aplikasi nyata, sebuah Job Portal dapat memiliki puluhan bahkan ratusan data lowongan pekerjaan. Menuliskan setiap komponen secara manual tentu tidak efisien.

React menyediakan fitur List Rendering untuk menampilkan banyak data secara otomatis menggunakan method `.map()`. Selain itu, React juga memiliki Conditional Rendering yang memungkinkan aplikasi menampilkan tampilan yang berbeda berdasarkan kondisi tertentu.

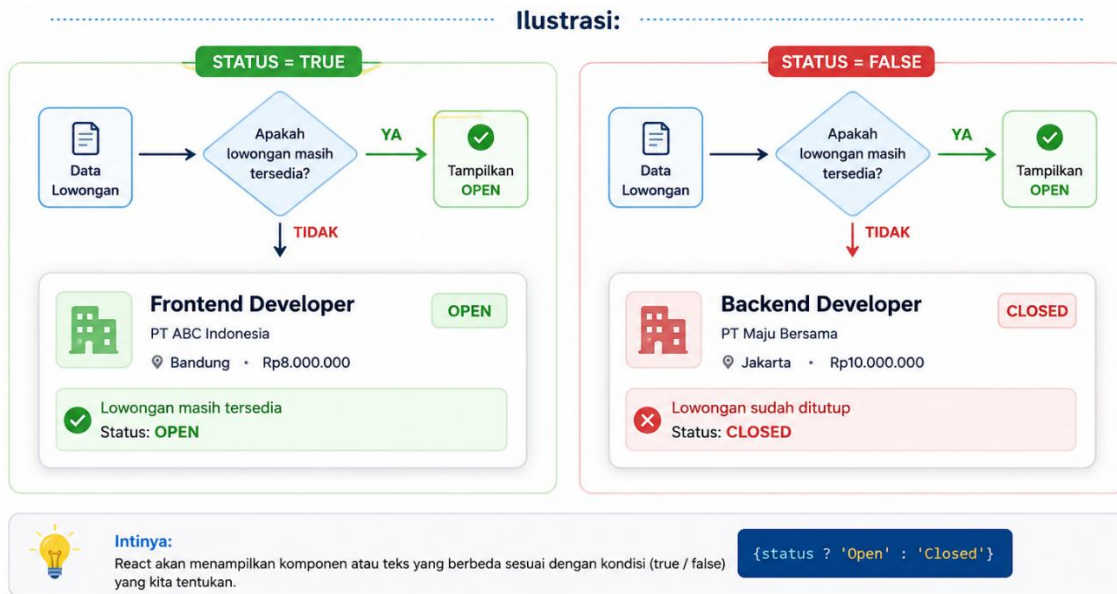
Pada bab ini, aplikasi CareerHub akan mulai menampilkan beberapa data lowongan pekerjaan sekaligus menggunakan `.map()` serta menampilkan status lowongan menggunakan Conditional Rendering.

6.2. Apa itu Conditional Rendering?

Conditional Rendering merupakan teknik untuk menampilkan suatu komponen berdasarkan kondisi tertentu.

Misalnya:

- Jika lowongan masih tersedia, tampilkan Open.
- Jika lowongan sudah ditutup, tampilkan Closed.



Menggunakan Operator Ternary

Conditional Rendering paling sering menggunakan operator ternary.

```

1  const isOpen = true;
2
3  return (
4    <h3>
5      {isOpen ? "Open" : "Closed"}
6    </h3>
7  );

```

Conditional Rendering pada CareerHub

Misalkan data lowongan memiliki status.

```

1  const job = {
2    title: "Frontend Developer",
3    status: true
4  }

```

Kemudian tampilkan status



```

1 <p>
2   Status :
3   {status ? " Open" : " Closed"}
4 </p>

```

Memberikan Warna Berdasarkan Status

Conditional Rendering juga dapat digunakan untuk mengubah tampilan.



```

1
2 <p
3   className={
4     status
5       ? "text-green-600 font-semibold"
6       : "text-red-600 font-semibold"
7   }
8 >
9   {status ? "Open" : "Closed"}
10 </p>

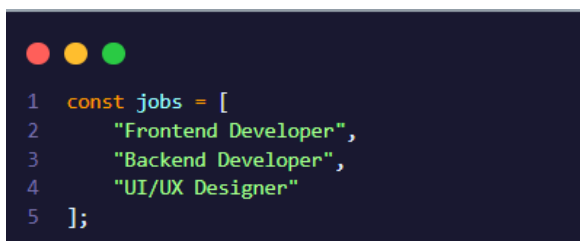
```

Jika status bernilai true, teks akan berwarna hijau. Jika false, teks akan berwarna merah.

6.3. Apa itu List Rendering?

List Rendering digunakan untuk menampilkan sekumpulan data menggunakan perulangan.

Misalnya terdapat data berikut.



```

1 const jobs = [
2   "Frontend Developer",
3   "Backend Developer",
4   "UI/UX Designer"
5 ];

```

Daripada menulis

```
1 <JobCard />
2
3 <JobCard />
4
5 <JobCard />
```

lebih baik menggunakan `.map()`.

Mengenal `.map()`

Method `.map()` digunakan untuk mengubah setiap data di dalam array menjadi elemen React.

Contoh sederhana.

```
1  const fruits = [
2    "Apel",
3    "Jeruk",
4    "Mangga"
5  ];
6
7  function App() {
8    return (
9      <>
10       {
11         fruits.map((fruit) => (
12           <p>{fruit}</p>
13         ))
14       }
15     </>
16   );
17 }
```

Property key

Saat menggunakan `.map()`, React meminta setiap data memiliki key.

Contoh.

```
1  {
2    jobs.map((job) => (
3      <JobCard
4        key={job.id}
5      />
6    ))
7  }
```

Property key membantu React mengenali setiap data sehingga proses rendering menjadi lebih efisien.

6.4.Studi Kasus

Langkah 1

Pada `App.jsx`, buat array data.

```
1  const jobs = [
2    {
3      id: 1,
4      title: "Frontend Developer",
5      company: "PT ABC Indonesia",
6      location: "Bandung",
7      salary: "Rp8.000.000",
8      status: true
9    },
10   {
11     id: 2,
12     title: "Backend Developer",
13     company: "PT Maju Bersama",
14     location: "Jakarta",
15     salary: "Rp10.000.000",
16     status: false
17   },
18   {
19     id: 3,
20     title: "UI/UX Designer",
21     company: "PT Creative Studio",
22     location: "Yogyakarta",
23     salary: "Rp7.500.000",
24     status: true
25   }
26 ];
```

Langkah 2

Ubah JobList.jsx.

```
1  const JobList = ({ jobs }) => {
2    return (
3      <section className="space-y-5">
4        {
5          jobs.map((job) => (
6            <JobCard
7              key={job.id}
8              title={job.title}
9              company={job.company}
10             location={job.location}
11             salary={job.salary}
12             status={job.status}
13           ) />
14         )
15       }
16     </section>
17   );
18 };
```

Langkah 3

Terima props baru pada JobCard.jsx.

```
1  const JobCard = ({
2    title,
3    company,
4    location,
5    salary,
6    status
7  }) => {
```

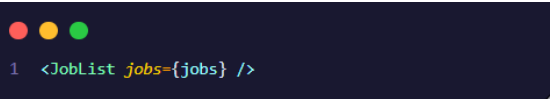
Langkah 4

Tambahkan status.

```
1  <p
2    className={
3      status
4        ? "text-green-600 font-semibold"
5        : "text-red-600 font-semibold"
6      }
7  >
8    Status :
9    {status ? " Open" : " Closed"}
10 </p>
```

Langkah 5

Kirim data dari App.jsx.



```
1 <JobList jobs={jobs} />
```

6.5. Ringkasan Bab

Pada bab ini peserta telah mempelajari cara menggunakan List Rendering untuk menampilkan sekumpulan data menggunakan method `.map()`, sehingga komponen `JobCard` tidak lagi ditulis secara manual satu per satu. Peserta juga memahami pentingnya penggunaan properti `key` agar React dapat mengelola proses rendering secara efisien.

Selain itu, peserta telah menerapkan Conditional Rendering untuk menampilkan status lowongan pekerjaan berdasarkan kondisi tertentu. Dengan menggabungkan kedua konsep tersebut, aplikasi `CareerHub` kini mampu menampilkan banyak data lowongan secara dinamis dengan tampilan yang berbeda sesuai status masing-masing.

BAB 7

MERESPONS EVENT DAN STATE MANAGEMENT

7.1. Pendahuluan

Pada bab sebelumnya, peserta telah berhasil menampilkan daftar lowongan pekerjaan menggunakan List Rendering serta menerapkan Conditional Rendering untuk menampilkan status lowongan.

Namun, aplikasi CareerHub masih bersifat statis karena belum dapat merespons interaksi dari pengguna. Sebagai contoh, ketika tombol Lihat Detail atau Lamar diklik, belum ada perubahan apa pun pada tampilan aplikasi.

Pada bab ini peserta akan mempelajari cara menangani interaksi pengguna menggunakan Event Handling serta menyimpan data sementara menggunakan State. Dengan kedua konsep tersebut, aplikasi React dapat memberikan pengalaman yang lebih interaktif.

7.2. Apa itu Event Handling?

Event Handling merupakan mekanisme untuk menangani aksi yang dilakukan oleh pengguna, seperti:

- Klik tombol.
- Mengisi input.
- Memilih menu.
- Mengarahkan kursor ke suatu elemen.

React menyediakan berbagai event yang mirip dengan JavaScript, namun penulisannya menggunakan camelCase.

Contoh:

HTML	React
onclick	onClick
onchange	onChange
onsubmit	onSubmit
onmouseover	onMouseOver

Contoh Event Handling

```
1  function App() {  
2  
3    const handleClick = () => {  
4      alert("Tombol berhasil diklik");  
5    }  
6  
7    return (  
8      <button onClick={handleClick}>  
9        Klik Saya  
10     </button>  
11  )  
12  
13 }
```

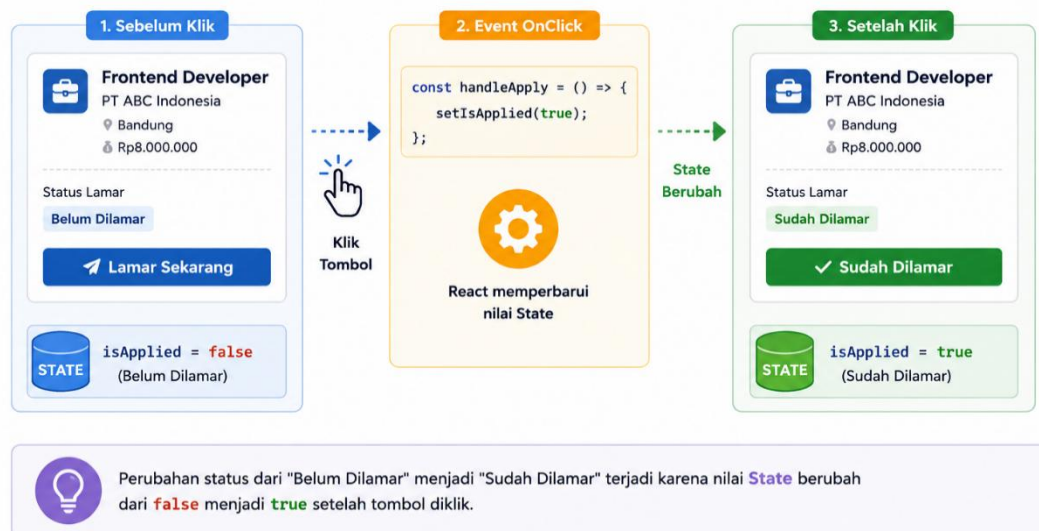
Ketika tombol diklik, fungsi handleClick() akan dijalankan.

7.3. Apa itu State?

State merupakan data yang dapat berubah selama aplikasi berjalan.

Berbeda dengan Props yang dikirim dari komponen induk, State dimiliki oleh komponen itu sendiri.

Sebagai contoh:



Perubahan tersebut disimpan menggunakan State.

useState

React menyediakan Hook bernama useState.

```
1 import { useState } from "react";
```

Penulisannya:

```
1 const [state, setState] = useState(nilaiAwal);
```

Contoh:

```
1 const [isApplied, setIsApplied] = useState(false);
```

Artinya:

- isApplied menyimpan nilai state.
- setIsApplied() digunakan untuk mengubah state.

Mengubah State

```
1 setIsApplied(true);
```

Setelah dijalankan React akan melakukan render ulang secara otomatis.

7.4.Studi Kasus

Langkah 1

Import useState.

```
1 import { useState } from "react";
```

Langkah 2

Tambahkan state.

```
1 const [isApplied, setIsApplied] = useState(false);
```

Langkah 3

Buat fungsi.

```
1 const handleApply = () => {  
2   setIsApplied(true);  
3 }
```

Langkah 4

Gunakan pada tombol.

```

1 <button
2   onClick={handleApply}
3 >
4   {
5     isApplied
6     ? "Sudah Dilamar"
7     : "Lamar Sekarang"
8   }
9 </button>

```

Langkah 6

Ubah warna tombol.

```

1 <button
2   onClick={handleApply}
3   disabled={isApplied}
4   className={`mt-5 w-full py-2 rounded-lg text-white transition
5   ${isApplied
6     ? "bg-green-600 cursor-not-allowed"
7     : "bg-blue-600 hover:bg-blue-700"}
8 >
9   {
10    isApplied
11    ? "✓ Sudah Dilamar"
12    : "Lamar Sekarang"
13  }
14 </button>

```

7.5. Ringkasan Bab

Pada bab ini peserta telah mempelajari cara menangani interaksi pengguna menggunakan Event Handling melalui atribut onClick serta memahami konsep State menggunakan Hook useState. Dengan memanfaatkan state, aplikasi React dapat menyimpan dan mengubah data yang bersifat dinamis sesuai dengan aksi pengguna.

Melalui studi kasus pada aplikasi CareerHub, peserta berhasil mengimplementasikan fitur Lamar Sekarang, di mana teks dan warna tombol berubah setelah pengguna melakukan aksi klik. Materi pada bab ini menjadi dasar penting untuk bab berikutnya, karena fitur seperti pencarian, filter, sorting, pagination, dan editing akan memanfaatkan konsep State yang sama untuk mengelola perubahan data dan tampilan aplikasi.

BAB 8

UJIAN TENGAH SEMESTER

(PROYEK SEDERHANA)

BAB 9

FILTER & PENCARIAN

9.1. Pendahuluan

Pada aplikasi yang memiliki banyak data, menampilkan seluruh data sekaligus dapat membuat pengguna kesulitan menemukan informasi yang dibutuhkan.

Misalnya, sebuah aplikasi memiliki 20 atau bahkan 100 produk. Pengguna mungkin hanya ingin mencari:

- Produk dengan nama tertentu.
- Produk dari kategori tertentu.
- Produk yang masih tersedia.
- Produk dengan harga tertentu.

Oleh karena itu, aplikasi perlu menyediakan fitur pencarian dan filter. Pada React, fitur tersebut dapat dibuat dengan memanfaatkan:

- State untuk menyimpan input pencarian.
- Event handling untuk menangkap perubahan input.
- Method `filter()` untuk menyaring data.
- Conditional rendering untuk menangani data yang tidak ditemukan.
- List rendering menggunakan `.map()` untuk menampilkan hasil.

Pada bab ini kita akan melanjutkan project sebelumnya dan menambahkan fitur Search dan Filter Produk.

9.2. Apa Itu Pencarian

Pencarian atau searching adalah proses untuk menemukan data berdasarkan kata kunci yang diberikan oleh pengguna.

Misalnya terdapat data:

```
const jobs = [
  {
    id: 1,
    jobName: "Fullstack Developer",
    company: "PT. ABC Indonesia",
    location: "Bandung",
    salary: "Rp. 8.000.000",
    status: true
  },
  {
    id: 2,
    jobName: "Backend Developer",
    company: "PT. DEF Indonesia",
    location: "Jakarta",
    salary: "Rp. 8.000.000",
    status: false
  },
  {
    id: 3,
    jobName: "Frontend Developer",
    company: "PT. GHI Indonesia",
    location: "Medan",
    salary: "Rp. 8.000.000",
    status: true
  }
];
```

Jika pengguna mengetik “Frontend” maka aplikasi hanya menampilkan:

```
{
  id: 3,
  jobName: "Frontend Developer",
  company: "PT. GHI Indonesia",
  location: "Medan",
  salary: "Rp. 8.000.000",
  status: true
}
```

9.3. Menenal Method filter()

JavaScript menyediakan method `.filter()` untuk menyaring data berdasarkan kondisi tertentu. Contoh sederhana:

```
const numbers = [10, 20, 30, 40, 50];

const result = numbers.filter(
  (number) => number > 25
);

console.log(result);
```

Hasil:

```
// [30, 40, 50]
```

Artinya, `.filter()` hanya mengambil data yang memenuhi kondisi.

9.4.filter() pada Data Produk

Sekarang kita terapkan pada data lowongan kerja. Misalnya kita ingin mencari semua lowongan yang berada di Bandung.

```
const bandungJobs = jobs.filter(
  (job) => job.location === "Bandung"
);

console.log(bandungJobs);
```

Hasilnya:

```
{
  id: 1,
  jobName: "Fullstack Developer",
  company: "PT. ABC Indonesia",
  location: "Bandung",
  salary: "Rp. 8.000.000",
  status: true
},
```

Kita juga dapat mencari lowongan yang masih tersedia berdasarkan status.

```
const availableJobs = jobs.filter(
  (job) => job.status === true
);
```

Hasilnya:

```
// Fullstack Developer  
// Frontend Developer
```

Karena:

```
Fullstack Developer → true  
// Backend Developer → false  
// Frontend Developer → true
```

9.5. Membuat Input Pencarian

Pada React, kita membutuhkan state untuk menyimpan keyword yang dimasukkan pengguna.

```
import { useState } from "react";  
  
const JobList = ({ jobs }) => {  
  const [search, setSearch] = useState("");  
  
  return (  
    <div>  
      <input  
        type="text"  
        placeholder="Cari lowongan..."  
        value={search}  
        onChange={(event) =>  
          setSearch(event.target.value)  
        }  
      />  
    </div>  
  );  
};  
  
export default JobList;
```

Bagian:

```
    onChange={(event) =>  
      setSearch(event.target.value)  
    }  
  }
```

berfungsi untuk mengambil nilai yang sedang diketik pengguna.

9.6. Menghubungkan Search dengan filter()

Sekarang kita gunakan state search untuk melakukan pencarian.

```
const filteredJobs = jobs.filter((job) =>  
  job.jobName  
    .toLowerCase()  
    .includes(search.toLowerCase())  
);
```

Misalnya: search = "Frontend"

maka React akan mencari:

```
// Fullstack Developer  
// Backend Developer  
// Frontend Developer
```

Dan menghasilkan

```
// Frontend Developer
```

Penggunaan toLowerCase() membuat pencarian tidak sensitif terhadap huruf besar dan kecil. Tanpa toLowerCase():

```
// Frontend  
// frontend  
// FRONTEND
```

Data diatas akan dianggap data yang berbeda.

9.7. Menampilkan Hasil Pencarian

Setelah mendapatkan data filteredJobs, kita dapat menampilkannya menggunakan .map().

```

    {filteredJobs.map((job) => (
      <div key={job.id}>
        <h3>{job.jobName}</h3>
        <p>{job.company}</p>
        <p>{job.location}</p>
        <p>{job.salary}</p>
      </div>
    ))}

```

9.8. Conditional Rendering pada Hasil Pencarian

Bagaimana jika pengguna mencari pekerjaan yang tidak tersedia? Misalnya mencari “Data Analyst” sementara data tersebut tidak ada. Kita dapat menggunakan conditional rendering:

```

{filteredJobs.length > 0 ? (
  filteredJobs.map((job) => (
    <JobCard
      key={job.id}
      job={job}
    />
  ))
) : (
  <p>
    Lowongan tidak ditemukan.
  </p>
)}

```

Dengan demikian, pengguna mendapatkan informasi yang jelas.

Selain pencarian berdasarkan nama pekerjaan, kita dapat membuat filter lokasi.

```
const [location, setLocation] = useState("Semua");
```

Kemudian

```

<select
  value={location}
  onChange={(event) =>
    setLocation(event.target.value)
  }

```

```

>
  <option value="Semua">
    Semua Lokasi
  </option>

  <option value="Bandung">
    Bandung
  </option>

  <option value="Jakarta">
    Jakarta
  </option>

  <option value="Medan">
    Medan
  </option>
</select>

```

Sekarang kita dapat menggabungkan pencarian berdasarkan nama pekerjaan dengan filter lokasi.

```

const filteredJobs = jobs.filter((job) => {

  const matchesSearch = job.jobName
    .toLowerCase()
    .includes(search.toLowerCase());

  const matchesLocation =
    location === "Semua" ||
    job.location === location;

  return matchesSearch && matchesLocation;
});

```

Kita juga dapat menggunakan property status.

Pada data kita status: true berarti lowongan masih tersedia. Sedangkan status: false berarti lowongan sudah tidak tersedia.

Kita dapat membuat:

```

const [status, setStatus] = useState("Semua");

```

Berikut full coding untuk implementasi search dan filter

```
import { useState } from "react";

const jobs = [
  {
    id: 1,
    jobName: "Fullstack Developer",
    company: "PT. ABC Indonesia",
    location: "Bandung",
    salary: "Rp. 8.000.000",
    status: true
  },
  {
    id: 2,
    jobName: "Backend Developer",
    company: "PT. DEF Indonesia",
    location: "Jakarta",
    salary: "Rp. 8.000.000",
    status: false
  },
  {
    id: 3,
    jobName: "Frontend Developer",
    company: "PT. GHI Indonesia",
    location: "Medan",
    salary: "Rp. 8.000.000",
    status: true
  }
];

const JobList = ({ jobs }) => {

  const [search, setSearch] = useState("");
  const [location, setLocation] = useState("Semua");
  const [status, setStatus] = useState("Semua");

  const filteredJobs = jobs.filter((job) => {

    const matchesSearch =
      job.jobName
        .toLowerCase()
        .includes(search.toLowerCase());

    const matchesLocation =
      location === "Semua" ||
      job.location === location;
  });
}
```

```

const matchesStatus =
  status === "Semua" ||
  (status === "Tersedia" && job.status === true) ||
  (status === "Ditutup" && job.status === false);

return (
  matchesSearch &&
  matchesLocation &&
  matchesStatus
);
});

return (
  <section className="space-y-6">

    <div className="flex flex-col md:flex-row gap-4">

      <input
        type="text"
        placeholder="Cari lowongan..."
        value={search}
        onChange={(event) =>
          setSearch(event.target.value)
        }
        className="flex-1 border rounded-xl px-4 py-3"
      />

      <select
        value={location}
        onChange={(event) =>
          setLocation(event.target.value)
        }
        className="border rounded-xl px-4 py-3"
      >
        <option value="Semua">
          Semua Lokasi
        </option>
        <option value="Bandung">
          Bandung
        </option>
        <option value="Jakarta">
          Jakarta
        </option>
        <option value="Medan">
          Medan
        </option>
      </select>
    </div>
  </section>
);

```

```

        <select
          value={status}
          onChange={(event) =>
            setStatus(event.target.value)
          }
          className="border rounded-xl px-4 py-3"
        >
          <option value="Semua">
            Semua Status
          </option>
          <option value="Tersedia">
            Tersedia
          </option>
          <option value="Ditutup">
            Ditutup
          </option>
        </select>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-6">

        {filteredJobs.length > 0 ? (

          filteredJobs.map((job) => (
            <div
              key={job.id}
              className="bg-white rounded-xl
shadow-md p-6"
            >
              <h3 className="text-xl font-bold">
                {job.jobName}
              </h3>

              <p className="text-gray-600">
                {job.company}
              </p>

              <p className="text-gray-500">
                📍 {job.location}
              </p>

              <p className="font-semibold mt-3">
                {job.salary}
              </p>
            </div>
          ))
        ) : (
          <p>No jobs found</p>
        )}
      </div>
    </div>
  )}

```

```

        <p className="mt-2">
          {job.status
            ? "🟢 Tersedia"
            : "🔴 Ditutup"
          }
        </p>
      </div>
    ))

    ) : (

      <p className="col-span-full text-center">
        Lowongan tidak ditemukan.
      </p>

    )}

  </div>

</section>
);
};

export default JobList;

```

9.9. Ringkasan Bab

Pada bab ini kita telah mempelajari:

- Konsep searching pada website pencarian kerja.
- Method .filter().
- Membuat input pencarian.
- Menggunakan useState() untuk search.
- Menggunakan .includes().
- Menggunakan .toLowerCase().
- Filter berdasarkan lokasi.
- Filter berdasarkan status.
- Menggabungkan beberapa filter.
- Conditional rendering ketika lowongan tidak ditemukan.
- Menampilkan hasil menggunakan .map().

BAB 10

SORTING

10.1. Pendahuluan

Setelah pengguna dapat mencari dan memfilter lowongan, fitur berikutnya yang dapat ditambahkan adalah sorting.

Sorting digunakan untuk menentukan urutan data yang ditampilkan kepada pengguna.

Pada website CareerHub, sorting dapat digunakan untuk mengurutkan lowongan berdasarkan:

- Nama pekerjaan A-Z.
- Nama pekerjaan Z-A.
- Perusahaan A-Z.
- Lokasi A-Z.
- Gaji terendah.
- Gaji tertinggi.

Dengan adanya sorting, pengguna dapat menemukan lowongan dengan lebih mudah.

10.2. Apa Itu Sorting?

Sorting adalah proses mengurutkan sekumpulan data berdasarkan aturan tertentu.

Misalnya:

- Backend Developer
- Frontend Developer
- Fullstack Developer

Jika menggunakan sorting A-Z, maka urutannya:

- Backend Developer
- Frontend Developer
- Fullstack Developer

Sedangkan Z-A:

- Fullstack Developer
- Frontend Developer
- Backend Developer

10.3. Mengenal sort()

JavaScript menyediakan method `.sort()`.

Contoh:

```
const numbers = [50, 20, 40, 10, 30];  
  
numbers.sort((a, b) => a - b);  
  
console.log(numbers);
```

Hasilnya:

```
// [10, 20, 30, 40, 50]
```

10.4. Implementasi Sorting

- Sorting nama pekerjaan

```
const sortedJobs = [...jobs].sort(  
  (a, b) =>  
    a.jobName.localeCompare(b.jobName)  
);
```

Hasil:

```
// Backend Developer  
// Frontend Developer  
// Fullstack Developer
```

Untuk Z-A:

```
const sortedJobs = [...jobs].sort(  
  (a, b) =>  
    b.jobName.localeCompare(a.jobName)  
);
```

- Sorting berdasarkan perusahaan

```
const sortedJobs = [...jobs].sort(  
  (a, b) =>  
    a.company.localeCompare(b.company)  
);
```

Hasil:

```
// PT. ABC Indonesia  
// PT. DEF Indonesia  
// PT. GHI Indonesia
```

- Sorting berdasarkan lokasi

```
const sortedJobs = [...jobs].sort(  
  (a, b) =>  
    a.location.localeCompare(b.location)  
);
```

Hasil:

```
// Bandung  
// Jakarta  
// Medan
```

- Sorting berdasarkan salary

Di sinilah contoh menjadi sedikit lebih menarik. Pada data kita, salary disimpan sebagai string, Jika kita langsung menggunakan:

```
a.salary - b.salary
```

hasilnya tidak akan sesuai karena salary bukan angka. Kita perlu mengubah string tersebut menjadi angka.

Kita dapat membuat function:

```
const getSalaryValue = (salary) => {  
  return Number(  
    salary  
      .replace("Rp. ", "")  
      .replaceAll(".", "")  
  );  
};
```

Sorting salary terendah

```
const sortedJobs = [...jobs].sort(  
  (a, b) =>  
    getSalaryValue(a.salary) -  
    getSalaryValue(b.salary)  
);
```

Sorting salary tertinggi

```
const sortedJobs = [...jobs].sort(  
  (a, b) =>  
    getSalaryValue(b.salary) -  
    getSalaryValue(a.salary)  
);
```

Berikut full coding untuk implementasi search, filter dan sorting

```
import { useState } from "react";  
  
const jobs = [  
  {
```

```

        id: 1,
        jobName: "Fullstack Developer",
        company: "PT. ABC Indonesia",
        location: "Bandung",
        salary: "Rp. 8.000.000",
        status: true
      },
      {
        id: 2,
        jobName: "Backend Developer",
        company: "PT. DEF Indonesia",
        location: "Jakarta",
        salary: "Rp. 8.000.000",
        status: false
      },
      {
        id: 3,
        jobName: "Frontend Developer",
        company: "PT. GHI Indonesia",
        location: "Medan",
        salary: "Rp. 8.000.000",
        status: true
      }
    ]
  };

const getSalaryValue = (salary) => {
  return Number(
    salary
      .replace("Rp. ", "")
      .replaceAll(".", "")
  );
};

const JobList = ({ jobs }) => {

  const [search, setSearch] = useState("");
  const [location, setLocation] = useState("Semua");
  const [status, setStatus] = useState("Semua");
  const [sortBy, setSortBy] = useState("default");

  const filteredJobs = jobs.filter((job) => {

    const matchesSearch =
      job.jobName
        .toLowerCase()
        .includes(search.toLowerCase());

    const matchesLocation =

```

```

        location === "Semua" ||
        job.location === location;

    const matchesStatus =
        status === "Semua" ||
        (status === "Tersedia" &&
            job.status === true) ||
        (status === "Ditutup" &&
            job.status === false);

    return (
        matchesSearch &&
        matchesLocation &&
        matchesStatus
    );
});

const sortedJobs = [...filteredJobs];

if (sortBy === "name-asc") {
    sortedJobs.sort(
        (a, b) =>
            a.jobName.localeCompare(b.jobName)
    );
}

if (sortBy === "name-desc") {
    sortedJobs.sort(
        (a, b) =>
            b.jobName.localeCompare(a.jobName)
    );
}

if (sortBy === "salary-low") {
    sortedJobs.sort(
        (a, b) =>
            getSalaryValue(a.salary) -
            getSalaryValue(b.salary)
    );
}

if (sortBy === "salary-high") {
    sortedJobs.sort(
        (a, b) =>
            getSalaryValue(b.salary) -
            getSalaryValue(a.salary)
    );
}

```

```

return (
  <section className="space-y-6">

    <div className="
      flex
      flex-col
      md:flex-row
      gap-4
    ">

      <input
        type="text"
        placeholder="Cari lowongan..."
        value={search}
        onChange={(event) =>
          setSearch(event.target.value)
        }
        className="
          flex-1
          border
          rounded-xl
          px-4
          py-3
        "
      />

      <select
        value={location}
        onChange={(event) =>
          setLocation(event.target.value)
        }
        className="
          border
          rounded-xl
          px-4
          py-3
        "
      >
        <option value="Semua">
          Semua Lokasi
        </option>

        <option value="Bandung">
          Bandung
        </option>

        <option value="Jakarta">

```

```

        Jakarta
      </option>

      <option value="Medan">
        Medan
      </option>
    </select>

    <select
      value={status}
      onChange={(event) =>
        setStatus(event.target.value)
      }
      className="
        border
        rounded-xl
        px-4
        py-3
      "
    >
      <option value="Semua">
        Semua Status
      </option>

      <option value="Tersedia">
        Tersedia
      </option>

      <option value="Ditutup">
        Ditutup
      </option>
    </select>

    <select
      value={sortBy}
      onChange={(event) =>
        setSortBy(event.target.value)
      }
      className="
        border
        rounded-xl
        px-4
        py-3
      "
    >
      <option value="default">
        Urutan Default
      </option>

```

```

      <option value="name-asc">
        Nama A-Z
      </option>

      <option value="name-desc">
        Nama Z-A
      </option>

      <option value="salary-low">
        Gaji Terendah
      </option>

      <option value="salary-high">
        Gaji Tertinggi
      </option>
    </select>

  </div>

  <p className="text-gray-500">
    Menampilkan {sortedJobs.length} lowongan
  </p>

  <div className="
    grid
    grid-cols-1
    md:grid-cols-2
    lg:grid-cols-3
    gap-6
  ">

    {sortedJobs.length > 0 ? (
      sortedJobs.map((job) => (
        <div
          key={job.id}
          className="
            bg-white
            rounded-2xl
            shadow-md
            p-6
            hover:shadow-xl
            transition
          "
        >

```

```

        <h3 className="
            text-xl
            font-bold
        ">
            {job.jobName}
        </h3>

        <p className="
            text-gray-600
            mt-2
        ">
            {job.company}
        </p>

        <p className="
            text-gray-500
            mt-1
        ">
            📍 {job.location}
        </p>

        <p className="
            font-semibold
            text-blue-600
            mt-4
        ">
            {job.salary}
        </p>

        <p className="mt-2">
            {job.status
                ? "🟢 Lowongan Tersedia"
                : "🟡 Lowongan Ditutup"
            }
        </p>
    </div>

    ))

    ) : (

        <div className="
            col-span-full
            text-center
            py-12
        ">

            <p className="

```

```

        text-gray-500
        text-lg
      ">
        Lowongan tidak ditemukan.
      </p>
    </div>

  })

</div>

</section>
);
};

export default JobList;

```

10.5. Ringkasan Bab

Pada bab ini kita telah mempelajari:

- Pengertian sorting.
- Method `.sort()`.
- Sorting ascending.
- Sorting descending.
- Sorting berdasarkan nama pekerjaan.
- Sorting berdasarkan perusahaan.
- Sorting berdasarkan lokasi.
- Sorting berdasarkan salary.
- Mengubah salary dari string menjadi angka.
- Penggunaan `localeCompare()`.
- Penggunaan Spread Operator sebelum `.sort()`.
- Membuat state sorting.
- Menggabungkan Search + Filter + Sorting.

BAB 11

PAGINATION

11.1. Pendahuluan

Pada bab sebelumnya kita telah mempelajari Filter, Pencarian, dan Sorting pada website CareerHub. Setelah pengguna dapat mencari dan mengurutkan lowongan, masalah berikutnya adalah bagaimana jika jumlah lowongan yang tersedia sangat banyak?

Misalnya CareerHub memiliki:

100 lowongan kerja

Jika seluruh data ditampilkan dalam satu halaman, pengguna harus melakukan scroll yang sangat panjang.

Untuk mengatasi masalah tersebut, kita dapat menggunakan Pagination. Pagination memungkinkan data dibagi menjadi beberapa halaman sehingga pengguna hanya melihat sebagian data pada satu waktu.

Contoh:

Total Lowongan = 30

- Halaman 1 → Data 1 - 6
- Halaman 2 → Data 7 - 12
- Halaman 3 → Data 13 - 18
- Halaman 4 → Data 19 - 24
- Halaman 5 → Data 25 - 30

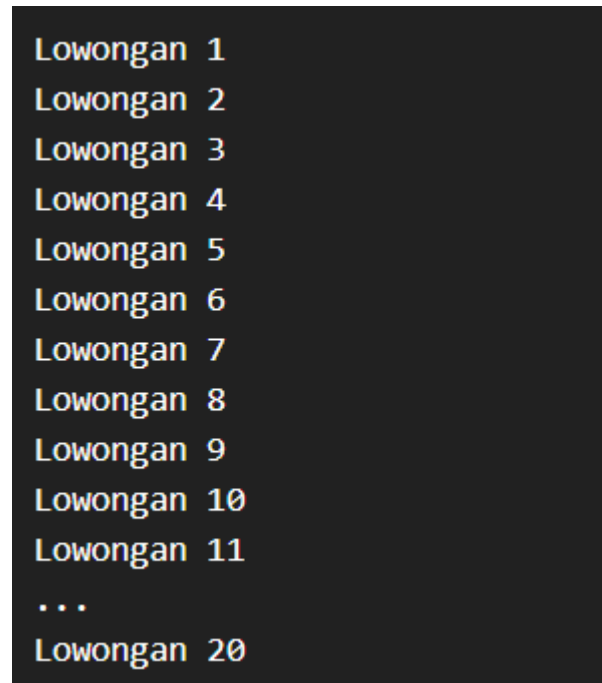
Pada bab ini kita akan membuat fitur pagination pada daftar lowongan CareerHub menggunakan React.

11.2. Apa Itu Pagination?

Pagination adalah teknik untuk membagi sekumpulan data menjadi beberapa halaman.

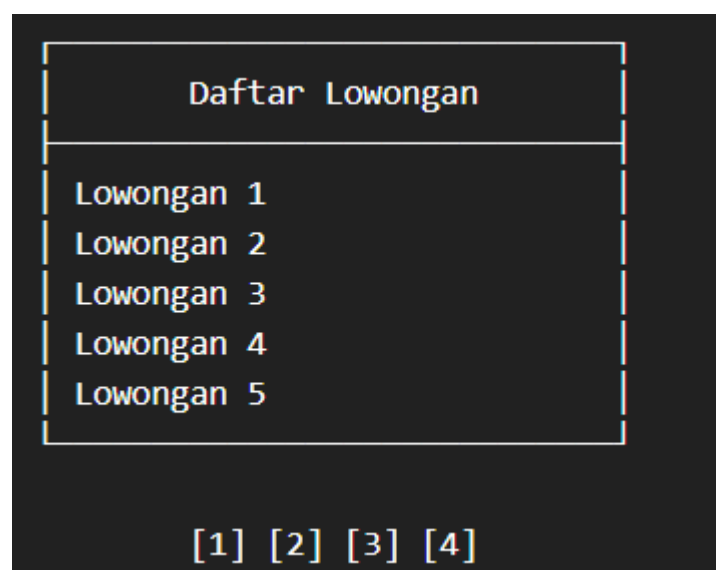
Sebagai contoh, CareerHub memiliki 20 lowongan.

Tanpa pagination:



```
Lowongan 1
Lowongan 2
Lowongan 3
Lowongan 4
Lowongan 5
Lowongan 6
Lowongan 7
Lowongan 8
Lowongan 9
Lowongan 10
Lowongan 11
...
Lowongan 20
```

Dengan pagination:



```
Daftar Lowongan
Lowongan 1
Lowongan 2
Lowongan 3
Lowongan 4
Lowongan 5
[1] [2] [3] [4]
```

Jika pengguna memilih halaman 2, maka data yang ditampilkan berubah.

11.3. Pagination pada CareerHub

Pada CareerHub, pagination akan digunakan untuk membagi daftar lowongan menjadi beberapa halaman.

Misalnya kita menentukan:

```
const jobsPerPage = 2;
```

Dengan data:

```
const jobs = [
  {
    id: 1,
    jobName: "Fullstack Developer",
    company: "PT. ABC Indonesia",
    location: "Bandung",
    salary: "Rp. 8.000.000",
    status: true
  },
  {
    id: 2,
    jobName: "Backend Developer",
    company: "PT. DEF Indonesia",
    location: "Jakarta",
    salary: "Rp. 8.000.000",
    status: false
  },
  {
    id: 3,
    jobName: "Frontend Developer",
    company: "PT. GHI Indonesia",
    location: "Medan",
    salary: "Rp. 8.000.000",
    status: true
  }
];
```

Maka:

```
Halaman 1
├─ Fullstack Developer
└─ Backend Developer

Halaman 2
└─ Frontend Developer
```

11.4. Konsep Dasar Pagination

Untuk membuat pagination, terdapat beberapa informasi yang perlu kita ketahui.

1. Current Page

Menentukan halaman yang sedang aktif.

```
const [currentPage, setCurrentPage] = useState(1);
```

Nilai awal:

```
currentPage = 1
```

Artinya pengguna sedang berada di halaman pertama.

2. Data Per Halaman

Menentukan jumlah data yang ditampilkan dalam satu halaman.

```
const jobsPerPage = 2;
```

Artinya setiap halaman hanya menampilkan maksimal 2 lowongan.

3. Total Halaman

Total halaman dapat dihitung menggunakan:

```
const totalPages = Math.ceil(  
  jobs.length / jobsPerPage  
);
```

Misalnya:

```
Jumlah data      = 7
```

```
Data per halaman = 3
```

Maka:

```
7 / 3 = 2.33
```

Karena halaman tidak mungkin menjadi 2.33, kita menggunakan Math.ceil():

```
Math.ceil(2.33) = 3
```

Jadi dibutuhkan 3 halaman.

11.5. Mengenal Math.ceil()

`Math.ceil()` digunakan untuk membulatkan angka ke atas.

Contoh:

```
Math.ceil(2.1);
```

Hasil:

```
3
```

Contoh lainnya:

```
Math.ceil(10 / 3);
```

Hasil:

```
4
```

Hal ini penting dalam pagination karena jumlah data tidak selalu habis dibagi oleh jumlah data per halaman.

11.6. Menentukan Data yang Ditampilkan

Setelah mengetahui halaman aktif, kita perlu menentukan data mana yang akan ditampilkan. Untuk melakukan hal tersebut, kita dapat menggunakan:

```
slice()
```

Contoh:

```
const startIndex =  
  (currentPage - 1) * jobsPerPage;  
  
const endIndex =  
  startIndex + jobsPerPage;  
  
const currentJobs =  
  jobs.slice(startIndex, endIndex);
```

Mari kita lihat prosesnya.

Jika:

```
currentPage = 1
```

```
jobsPerPage = 2
```

maka:

```
startIndex = (1 - 1) * 2;
```

hasil:

```
startIndex = 0
```

dan:

```
endIndex = 2
```

Sehingga:

```
jobs.slice(0, 2);
```

mengambil data:

```
Data 1
```

```
Data 2
```

11.7. Ketika Berpindah ke Halaman 2

Misalnya:

```
currentPage = 2
```

```
jobsPerPage = 2
```

Maka:

```
startIndex = (2 - 1) * 2;
```

hasil:

```
startIndex = 2
```

dan:

```
endIndex = 4
```

Sehingga:

```
jobs.slice(2, 4);
```

akan mengambil:

Data 3

Data 4

Jadi:

Halaman 1

→ Data 1 - 2

Halaman 2

→ Data 3 - 4

Halaman 3

→ Data 5 - 6

11.8. Membuat State Halaman

Pada React kita membutuhkan `useState()` untuk menyimpan halaman yang sedang aktif.

```
import { useState } from "react";

const JobList = ({ jobs }) => {

  const [currentPage, setCurrentPage] =
    useState(1);

  const jobsPerPage = 2;

  return (
```

```

        <div>
          { /* Daftar Job */ }
        </div>
      );
    };
  };
  export default JobList;

```

Ketika pengguna memilih halaman lain, kita akan mengubah:

```
setCurrentPage()
```

11.9. Mengambil Data untuk Halaman Aktif

Selanjutnya:

```

const startIndex =
  (currentPage - 1) * jobsPerPage;

const endIndex =
  startIndex + jobsPerPage;

const currentJobs =
  jobs.slice(startIndex, endIndex);

```

Kemudian kita tampilkan menggunakan `.map()`:

```

{currentJobs.map((job) => (
  <div key={job.id}>
    <h3>{job.jobName}</h3>

    <p>{job.company}</p>

    <p>{job.location}</p>

    <p>{job.salary}</p>
  </div>
))}

```

Sekarang hanya data pada halaman aktif yang akan ditampilkan.

11.10. Membuat Tombol Pagination

Kita dapat membuat tombol:

```

<button
  onClick={() => setCurrentPage(1)}
>
  1
</button>

<button
  onClick={() => setCurrentPage(2)}
>
  2
</button>

```

Namun cara tersebut kurang fleksibel. Bagaimana jika terdapat 10 halaman? Kita tentu tidak ingin menuliskan:

```

<button>1</button>

<button>2</button>

<button>3</button>

...

<button>10</button>

```

Oleh karena itu, kita dapat membuat tombol pagination secara dinamis menggunakan `map()`.

11.11. Membuat Nomor Halaman Secara Dinamis

Pertama kita hitung jumlah halaman:

```

const totalPages = Math.ceil(
  jobs.length / jobsPerPage
);

```

Kemudian membuat array halaman:

```

const pageNumbers = Array.from(
  { length: totalPages },
  (_, index) => index + 1
);

```

Jika:

```
totalPages = 3
```

maka:

```
pageNumbers
```

akan menghasilkan:

```
[1, 2, 3]
```

11.12. Menampilkan Tombol dengan .map()

```
<div className="flex gap-2">

  {pageNumbers.map((page) => (
    <button
      key={page}
      onClick={() =>
        setCurrentPage(page)
      }
    >
      {page}
    </button>
  ))}

</div>
```

Hasilnya:

```
[ 1 ] [ 2 ] [ 3 ]
```

Ketika pengguna mengklik:

```
[2]
```

maka:

```
currentPage
```

akan berubah menjadi:

```
2
```

React akan melakukan render ulang dan menampilkan data halaman kedua.

11.13. Memberikan Style pada Halaman Aktif

Kita dapat membuat halaman aktif terlihat berbeda menggunakan conditional rendering dan Tailwind CSS.

```
<button
  key={page}
  onClick={() => setCurrentPage(page)}
  className={`
    px-4
    py-2
    rounded-lg
    ${
      currentPage === page
        ? "bg-blue-600 text-white"
        : "bg-gray-100 text-gray-700"
    }
  `}
>
  {page}
</button>
```

Jika halaman aktif adalah halaman 2:

```
[ 1 ] [ 2 ] [ 3 ]
      ↑
    aktif
```

Maka tombol halaman 2 akan memiliki tampilan berbeda.

11.14. Tombol Previous dan Next

Selain nomor halaman, pagination biasanya memiliki tombol Previous dan Next. Tombol Previous digunakan untuk berpindah ke halaman sebelumnya, sedangkan tombol Next digunakan untuk berpindah ke halaman berikutnya.

Contohnya:

```
[ Previous ] [ 1 ] [ 2 ] [ 3 ] [ Next ]
```

Jika pengguna berada di halaman 2 dan menekan Previous, maka pengguna akan berpindah ke halaman 1.

Jika menekan Next, maka pengguna akan berpindah ke halaman 3.

Membuat Tombol Previous

Kita dapat menggunakan:

```
<button
  onClick={() => setCurrentPage(currentPage - 1)}
>
  Previous
</button>
```

Namun terdapat masalah.

Jika pengguna sedang berada di halaman pertama:

```
currentPage = 1
```

Kemudian tombol Previous ditekan, maka:

```
currentPage = 0
```

Padahal halaman 0 tidak tersedia.

Oleh karena itu, kita perlu memberikan kondisi.

```
<button
  onClick={() => setCurrentPage(currentPage - 1)}
  disabled={currentPage === 1}
>
  Previous
</button>
```

Dengan kondisi tersebut, tombol Previous akan dinonaktifkan ketika pengguna berada di halaman pertama.

Membuat Tombol Next

Untuk tombol Next:

```
<button
  onClick={() => setCurrentPage(currentPage + 1)}
>
  Next
</button>
```

Sama seperti Previous, kita juga harus mencegah pengguna berpindah ke halaman yang tidak tersedia.

Misalnya total halaman hanya 3.

Jika pengguna berada di:

```
currentPage = 3
```

Kemudian menekan Next, maka:

```
currentPage = 4
```

Padahal halaman 4 tidak tersedia.

Oleh karena itu:

```
<button
  onClick={() => setCurrentPage(currentPage + 1)}
  disabled={currentPage === totalPages}
>
  Next
</button>
```

Dengan demikian, tombol Next akan dinonaktifkan ketika pengguna sudah berada di halaman terakhir.

Contoh Lengkap Previous dan Next

```
<div className="flex items-center gap-2">

  <button
    onClick={() => setCurrentPage(currentPage - 1)}
    disabled={currentPage === 1}
    className="px-4 py-2 rounded-lg bg-gray-200 disabled:opacity-50"
  >
```

```

        Previous
      </button>

      {pageNumbers.map((page) => (
        <button
          key={page}
          onClick={() => setCurrentPage(page)}
          className={`px-4 py-2 rounded-lg ${
            currentPage === page
              ? "bg-blue-600 text-white"
              : "bg-gray-100 text-gray-700"
            }`}
        >
          {page}
        </button>
      ))}

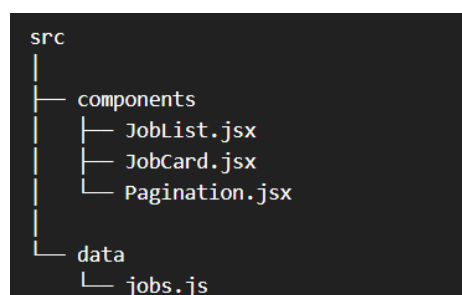
      <button
        onClick={() => setCurrentPage(currentPage + 1)}
        disabled={currentPage === totalPages}
        className="px-4 py-2 rounded-lg bg-gray-200 disabled:opacity-50"
      >
        Next
      </button>
    </div>

```

11.15. Membuat Komponen Pagination

Sampai tahap ini, seluruh kode pagination masih berada di dalam komponen JobList. Jika aplikasi semakin besar, kode tersebut dapat menjadi panjang. Salah satu solusi yang dapat digunakan adalah membuat komponen khusus: **Pagination.jsx**

Komponen tersebut bertugas menangani tampilan pagination. Sedangkan JobList bertugas menampilkan data pekerjaan. Struktur sederhananya:



Membuat Pagination.jsx

```
const Pagination = ({
  currentPage,
  totalPages,
  onPageChange
}) => {

  const pageNumbers = Array.from(
    { length: totalPages },
    (_, index) => index + 1
  );

  return (
    <div className="flex justify-center items-center gap-2 mt-6">

      <button
        onClick={() => onPageChange(currentPage - 1)}
        disabled={currentPage === 1}
        className="
          px-4 py-2
          rounded-lg
          border
          disabled:opacity-40
          disabled:cursor-not-allowed
        "
      >
        Previous
      </button>

      {pageNumbers.map((page) => (
        <button
          key={page}
          onClick={() => onPageChange(page)}
          className={`
            px-4 py-2
            rounded-lg
            ${
              currentPage === page
                ? "bg-blue-600 text-white"
                : "bg-gray-100 text-gray-700"
            }
          `}
        >
          {page}
        </button>
      ))}

    </div>
  );
}
```

```

        <button
          onClick={() => onPageChange(currentPage + 1)}
          disabled={currentPage === totalPages}
          className="
            px-4 py-2
            rounded-lg
            border
            disabled:opacity-40
            disabled:cursor-not-allowed
          "
        >
          Next
        </button>
      </div>
    );
  };
};

export default Pagination;

```

Pada komponen tersebut terdapat beberapa props:

currentPage

totalPages

onPageChange

currentPage

Digunakan untuk mengetahui halaman yang sedang aktif.

totalPages

Digunakan untuk mengetahui jumlah halaman yang tersedia.

onPageChange

Digunakan untuk mengirim perubahan halaman kembali ke komponen JobList.

Menggunakan Pagination pada JobList

Selanjutnya kita dapat menggunakan komponen tersebut pada JobList.

```

import { useState } from "react";
import Pagination from "../Pagination";

const JobList = ({ jobs }) => {

```

```

const [currentPage, setCurrentPage] = useState(1);

const jobsPerPage = 2;

const totalPages = Math.ceil(
  jobs.length / jobsPerPage
);

const startIndex =
  (currentPage - 1) * jobsPerPage;

const endIndex =
  startIndex + jobsPerPage;

const currentJobs =
  jobs.slice(startIndex, endIndex);

return (
  <div>

    <div className="grid gap-4">

      {currentJobs.map((job) => (
        <div
          key={job.id}
          className="p-5 border rounded-xl"
        >
          <h3 className="text-xl font-bold">
            {job.jobName}
          </h3>

          <p>{job.company}</p>

          <p>{job.location}</p>

          <p>{job.salary}</p>
        </div>
      ))}

    </div>

    <Pagination
      currentPage={currentPage}
      totalPages={totalPages}
      onPageChange={setCurrentPage}
    />
  </div>
)

```

```

    </div>
  );
};

export default JobList;

```

Perhatikan bagian:

```

<Pagination
  currentPage={currentPage}
  totalPages={totalPages}
  onPageChange={setCurrentPage}
/>

```

Data dari JobList dikirimkan ke komponen Pagination menggunakan props. Ketika pengguna menekan nomor halaman, Pagination menjalankan:

onPageChange(page)

Karena onPageChange berisi:

setCurrentPage

maka nilai currentPage akan berubah.

React kemudian melakukan re-render sehingga data yang ditampilkan juga berubah.

11.16. Ringkasan

Pada bab ini kita telah mempelajari:

- Pengertian pagination.
- Pagination pada CareerHub.
- Konsep dasar pagination.
- Penggunaan Math.ceil().
- Menentukan data yang ditampilkan menggunakan slice().
- Membuat state halaman menggunakan useState().
- Mengambil data berdasarkan halaman aktif.
- Membuat tombol pagination.
- Membuat nomor halaman secara dinamis menggunakan Array.from().

- Menampilkan tombol halaman menggunakan `.map()`.
- Membuat tombol Previous dan Next.
- Membuat komponen Pagination.

BAB 12

EDITING

12.1. Pendahuluan

Pada bab sebelumnya kita telah mempelajari bagaimana menampilkan data lowongan kerja CareerHub menggunakan Pagination. Saat ini pengguna sudah dapat melihat berbagai lowongan kerja yang tersedia. Namun, dalam sebuah aplikasi, data tidak selalu bersifat tetap. Terkadang terdapat informasi yang perlu diperbaiki atau diperbarui.

Misalnya terdapat data:

```
{
  id: 1,
  jobName: "Fullstack Developer",
  company: "PT. ABC Indonesia",
  location: "Bandung",
  salary: "Rp. 8.000.000",
  status: true
}
```

Kemudian perusahaan mengubah informasi gaji menjadi:

Rp. 10.000.000

Kita tentu tidak ingin membuat data baru hanya untuk mengubah informasi tersebut. Kita membutuhkan fitur Edit. Dengan fitur editing, pengguna dapat:

- memilih data yang ingin diubah
- menampilkan data tersebut pada form
- mengubah informasi
- menyimpan perubahan
- menampilkan data yang sudah diperbarui

Pada bab ini kita akan membuat fitur Editing Data Lowongan Kerja menggunakan React.

12.2. Apa Itu Editing

Editing adalah proses mengubah atau memperbarui data yang sudah tersedia. Sebagai contoh, CareerHub memiliki data:

Fullstack Developer

PT. ABC Indonesia

Bandung

Rp. 8.000.000

Kemudian pengguna ingin mengubah lokasi:

Bandung

menjadi:

Jakarta

Setelah proses editing dilakukan, data menjadi:

Fullstack Developer

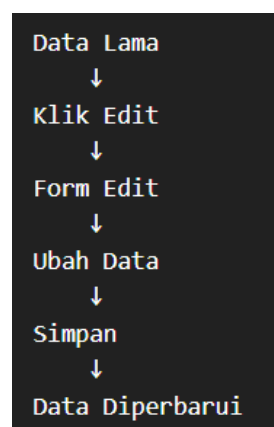
PT. ABC Indonesia

Jakarta

Rp. 8.000.000

Jadi, editing tidak membuat data baru, tetapi memperbarui data yang sudah ada.

Secara sederhana:



12.3. Menambahkan Tombol Edit

Sebelum membuat form editing, kita membutuhkan tombol Edit pada setiap data lowongan.

Misalnya kita memiliki komponen JobCard:

```
const JobCard = ({ job, onEdit }) => {  
  
  return (  
    <div className="border rounded-xl p-5">  
  
      <h3 className="text-xl font-bold">  
        {job.jobName}  
      </h3>  
  
      <p>{job.company}</p>  
  
      <p>{job.location}</p>  
  
      <p>{job.salary}</p>  
  
      <button  
        onClick={() => onEdit(job)}  
        className="mt-4 px-4 py-2 rounded-lg bg-blue-600 text-white"  
      >  
        Edit  
      </button>  
  
    </div>  
  );  
};  
  
export default JobCard;
```

Pada tombol Edit terdapat:

```
onClick={() => onEdit(job)}
```

Artinya ketika tombol Edit diklik, data job akan dikirim ke fungsi onEdit.

Sebagai contoh, jika pengguna mengklik Edit pada:

```
Fullstack Developer
```

maka seluruh data lowongan tersebut akan dikirim.

```
{
  id: 1,
  jobName: "Fullstack Developer",
  company: "PT. ABC Indonesia",
  location: "Bandung",
  salary: "Rp. 8.000.000",
  status: true
}
```

Data tersebut kemudian dapat digunakan untuk mengisi form editing.

12.4. Membuat State Untuk Data Yang Diedit

React membutuhkan state untuk menyimpan data yang sedang diedit.

Kita dapat menggunakan:

```
const [editingJob, setEditingJob] = useState(null);
```

Nilai awalnya adalah:

```
null
```

Artinya belum ada data yang sedang diedit.

Ketika pengguna menekan tombol Edit:

```
setEditingJob(job);
```

Maka state akan berisi data yang dipilih.

Contohnya:

```
{
  id: 1,
  jobName: "Fullstack Developer",
  company: "PT. ABC Indonesia",
  location: "Bandung",
  salary: "Rp. 8.000.000",
  status: true
}
```

Kita dapat menggunakan state tersebut untuk menentukan apakah form editing perlu ditampilkan.

```
{editingJob && (  
  <EditForm job={editingJob} />  
)}
```

Artinya:

Jika:

```
editingJob = null
```

maka form tidak ditampilkan.

Jika:

```
editingJob memiliki data
```

maka form editing ditampilkan.

12.5. Membuat Form Editing

Selanjutnya kita membuat komponen EditForm.

```
const EditForm = ({ job, onSave, onCancel }) => {  
  return (  
    <div className="p-6 border rounded-xl">  
      <h2 className="text-xl font-bold mb-4">  
        Edit Lowongan  
      </h2>  
      <div className="space-y-4">  
        <div>  
          <label>  
            Nama Pekerjaan  
          </label>  
          <input  
            type="text"  
            value={job.jobName}  
            className="w-full border rounded-lg px-4 py-2"  
          />  
        </div>  
      </div>  
    </div>  
  )  
}
```

```

        <label>
          Perusahaan
        </label>

        <input
          type="text"
          value={job.company}
          className="w-full border rounded-lg px-4 py-2"
        />
      </div>

      <div>
        <label>
          Lokasi
        </label>

        <input
          type="text"
          value={job.location}
          className="w-full border rounded-lg px-4 py-2"
        />
      </div>

      <div>
        <label>
          Salary
        </label>

        <input
          type="text"
          value={job.salary}
          className="w-full border rounded-lg px-4 py-2"
        />
      </div>
    </div>
  </div>
);
};

export default EditForm;

```

Pada tahap ini form sudah dapat menampilkan data yang dipilih. Namun terdapat satu masalah. Input tersebut belum dapat diubah karena value langsung mengambil data dari job. Kita membutuhkan state khusus untuk menyimpan perubahan form.

12.6. Menggunakan State Pada Form Editing

Kita dapat membuat state untuk menyimpan data form:

```
const [formData, setFormData] = useState({
  jobName: "",
  company: "",
  location: "",
  salary: "",
  status: true
});
```

Ketika data yang akan diedit diterima, kita dapat mengisi formData.

```
useEffect(() => {
  if (job) {
    setFormData({
      jobName: job.jobName,
      company: job.company,
      location: job.location,
      salary: job.salary,
      status: job.status
    });
  }
}, [job]);
```

Dengan demikian, ketika pengguna memilih suatu data, data tersebut akan masuk ke form.

Contohnya:

Nama Pekerjaan

[Fullstack Developer]

Perusahaan

[PT. ABC Indonesia]

Lokasi

[Bandung]

Salary

[Rp. 8.000.000]

Pengguna kemudian dapat mengubahnya.

Mengatur Perubahan Input

Kita dapat menggunakan onChange.

```
<input
  type="text"
  value={formData.jobName}
  onChange={(e) =>
    setFormData({
      ...formData,
      jobName: e.target.value
    })
  }
/>
```

Ketika pengguna mengubah input:

Fullstack Developer

menjadi:

Backend Developer

maka `formData.jobName` juga akan berubah.

Perhatikan penggunaan:

`...formData`

Spread Operator digunakan agar data lainnya tetap dipertahankan.

Misalnya:

```
{
  jobName: "Fullstack Developer",
  company: "PT. ABC Indonesia",
  location: "Bandung"
}
```

Kemudian jobName diubah.

```
setFormData({
  ...formData,
  jobName: "Backend Developer"
});
```

Hasilnya:

```
{
  jobName: "Backend Developer",
  company: "PT. ABC Indonesia",
  location: "Bandung"
}
```

Hanya jobName yang berubah, sedangkan data lainnya tetap.

12.7. Menyimpan Perubahan Data

Setelah pengguna selesai mengubah data, kita membutuhkan tombol Simpan.

```
<button
  onClick={handleSubmit}
  className="px-4 py-2 bg-blue-600 text-white rounded-lg"
>
  Simpan
</button>
```

Kemudian kita membuat fungsi:

```
const handleSubmit = () => {

  const updatedJob = {
    ...job,
    ...formData
  };

  onSave(updatedJob);
};
```

Perhatikan:

```
{
  ...job,
  ...formData
}
```

Data lama akan digabungkan dengan data baru.

Misalnya data awal:

```
{
  id: 1,
```

```
    jobName: "Fullstack Developer",
    company: "PT. ABC Indonesia",
    location: "Bandung",
    salary: "Rp. 8.000.000",
    status: true
  }
}
```

Kemudian pengguna mengubah:

```
location = Jakarta

salary = Rp. 10.000.000
```

Maka hasil akhirnya:

```
{
  id: 1,
  jobName: "Fullstack Developer",
  company: "PT. ABC Indonesia",
  location: "Jakarta",
  salary: "Rp. 10.000.000",
  status: true
}
```

id tetap digunakan untuk mengenali data yang sedang diperbarui.

12.8. Memperbaharui Data Map()

Setelah form selesai, kita perlu memperbarui data jobs. Kita dapat menggunakan `.map()`.

Misalnya state data jobs:

```
const [jobList, setJobList] = useState(jobs);
```

Kemudian:

```
const handleUpdateJob = (updatedJob) => {
  setJobList(
    jobList.map((job) =>
      job.id === updatedJob.id
        ? updatedJob
        : job
    )
  )
}
```

```
);  
  
    setEditingJob(null);  
};
```

Perhatikan bagian:

```
job.id === updatedJob.id
```

Digunakan untuk mencari data yang memiliki id yang sama.

Jika id sama:

```
? updatedJob
```

maka data diganti dengan data baru.

Jika id berbeda:

```
: job
```

maka data tetap menggunakan data lama.

Contoh Sederhana

Misalnya terdapat:

```
[  
  {  
    id: 1,  
    jobName: "Fullstack Developer"  
  },  
  {  
    id: 2,  
    jobName: "Backend Developer"  
  },  
  {  
    id: 3,  
    jobName: "Frontend Developer"  
  }  
]
```

Kemudian kita mengubah data:

```
id = 2
```

menjadi:

Senior Backend Developer

Dengan:

```
jobList.map((job) =>
  job.id === updatedJob.id
    ? updatedJob
    : job
)
```

hasilnya:

```
[
  {
    id: 1,
    jobName: "Fullstack Developer"
  },
  {
    id: 2,
    jobName: "Senior Backend Developer"
  },
  {
    id: 3,
    jobName: "Frontend Developer"
  }
]
```

Data lainnya tidak berubah.

Sekarang kita gabungkan seluruh konsep menjadi fitur editing sederhana.

Contoh JobList.jsx:

```
import { useState } from "react";
import JobCard from "../JobCard";
import EditForm from "../EditForm";

const JobList = ({ jobs }) => {

  const [jobList, setJobList] = useState(jobs);

  const [editingJob, setEditingJob] = useState(null);

  const handleEdit = (job) => {
    setEditingJob(job);
  };

  const handleUpdateJob = (updatedJob) => {
```

```

    setJobList(
      jobList.map((job) =>
        job.id === updatedJob.id
          ? updatedJob
          : job
      )
    );

    setEditingJob(null);
  };

  return (
    <div>

      <div className="grid gap-4">

        {jobList.map((job) => (
          <JobCard
            key={job.id}
            job={job}
            onEdit={handleEdit}
          />
        ))}

      </div>

      {editingJob && (
        <EditForm
          job={editingJob}
          onSave={handleUpdateJob}
          onCancel={() => setEditingJob(null)}
        />
      )}

    </div>
  );
};

export default JobList;

```

12.9. Ringkasan

Pada bab ini kita telah mempelajari:

- Pengertian editing.
- Membuat tombol Edit.

- Membuat state untuk data yang sedang diedit.
- Membuat form editing.
- Menggunakan `useEffect()` untuk mengisi form.
- Mengelola perubahan input menggunakan `onChange`.
- Menggunakan Spread Operator untuk memperbarui data.
- Membuat tombol Simpan.
- Memperbarui data menggunakan `.map()`.
- Menggunakan `id` untuk menentukan data yang akan diperbarui.
- Membuat komponen form editing.
- Mengimplementasikan fitur editing pada CareerHub.

BAB 13

CONTEXT API

13.1. Pendahuluan

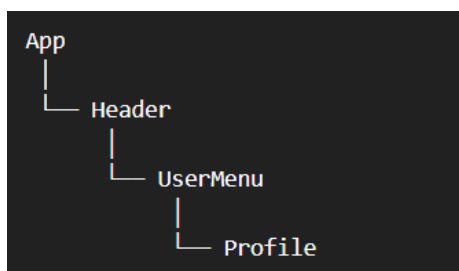
Pada bab sebelumnya kita telah mempelajari bagaimana membuat fitur Editing pada aplikasi CareerHub.

Dalam aplikasi React, data sering kali perlu digunakan oleh beberapa komponen yang berbeda. Misalnya, informasi pengguna yang sedang login dapat dibutuhkan oleh:

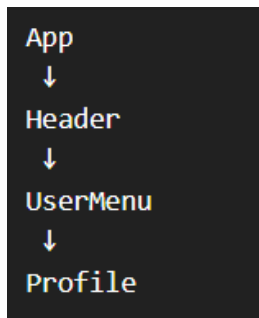
- Header
- Profile
- JobList
- Applications
- Footer

Jika data tersebut dikirim dari satu komponen ke komponen lain menggunakan props, kita harus mengirimkannya melewati beberapa komponen.

Contohnya:



Jika Profile membutuhkan data user dari App, maka data tersebut harus dikirim melalui:

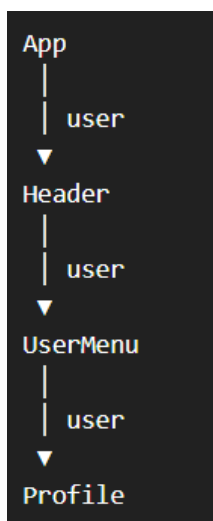


Padahal **Header** dan **UserMenu** mungkin tidak membutuhkan data tersebut. Kondisi seperti ini disebut Prop Drilling. React menyediakan Context API untuk membantu mengatasi masalah tersebut. Dengan Context API, data dapat disimpan pada sebuah context dan digunakan langsung oleh komponen yang membutuhkannya tanpa harus mengirim props melewati banyak komponen.

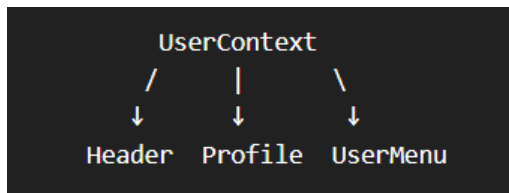
13.2. Mengenal Context API

Context API adalah fitur bawaan React yang memungkinkan data dibagikan kepada beberapa komponen tanpa harus meneruskan data tersebut melalui props secara manual pada setiap tingkat komponen.

Tanpa Context API:



Dengan Context API:



Komponen yang membutuhkan data dapat langsung mengambil data dari context.

Contoh Data yang Cocok Menggunakan Context

Context biasanya cocok digunakan untuk data yang dibutuhkan oleh banyak komponen.

Contohnya:

- User
- Theme
- Language
- Authentication
- Favorite Jobs
- Shopping Cart

Pada CareerHub, kita dapat menggunakan Context API untuk menyimpan informasi pengguna.

Misalnya:

```
const user = {
  name: "Wahyu",
  email: "wahyu@example.com"
};
```

Data tersebut dapat digunakan oleh:

- Header
- Profile
- Applications

tanpa harus mengirimkan user melalui props satu per satu.

13.3. Membuat Context

Untuk membuat Context, kita dapat menggunakan `createContext()`.

Pertama buat file:

```
src
├── context
│   └── UserContext.jsx
```

Kemudian:

```
import { createContext } from "react";

const UserContext = createContext();

export default UserContext;
```

`createContext()` digunakan untuk membuat sebuah context yang nantinya dapat menyimpan data yang ingin dibagikan.

Memberikan Nilai Default

Kita juga dapat memberikan nilai awal pada context.

```
import { createContext } from "react";

const UserContext = createContext({
  name: "",
  email: ""
});

export default UserContext;
```

Namun, biasanya data akan diberikan melalui **Provider**.

13.4. Membuat Provider

Setelah membuat context, kita perlu membuat Provider.

Provider digunakan untuk menyediakan data kepada komponen-komponen yang berada di dalamnya.

Contohnya:

```
import { createContext, useState } from "react";

export const UserContext = createContext();

const UserProvider = ({ children }) => {

  const [user, setUser] = useState({
    name: "Wahyu",
    email: "wahyu@example.com"
  });

  return (
    <UserContext.Provider
      value={{
        user,
        setUser
      }}
    >
      {children}
    </UserContext.Provider>
  );
};

export default UserProvider;
```

Pada kode tersebut:

<UserContext.Provider>

digunakan untuk menyediakan data.

Data yang diberikan terdapat pada:

```
value={{
  user,
  setUser
}}
```

Artinya komponen yang menggunakan UserContext dapat mengakses:

user

setUser

13.5. Menggunakan Provider Pada CareerHub

Setelah membuat UserProvider, kita perlu memasangnya pada komponen utama aplikasi.

Misalnya pada `App.jsx`:

```
import UserProvider from "../context/UserContext";
import Header from "../components/Header";
import JobList from "../components/JobList";

function App() {
  return (
    <UserProvider>

      <Header />

      <main>
        <JobList />
      </main>

    </UserProvider>
  );
}

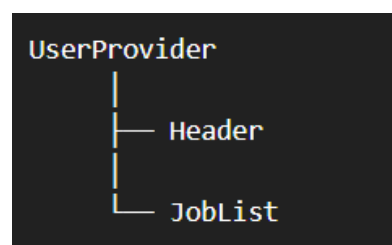
export default App;
```

Sekarang Header dan JobList berada di dalam:

`<UserProvider>`

Sehingga keduanya dapat mengakses data yang disediakan oleh UserContext.

Strukturnya:



Seluruh komponen yang berada di dalam UserProvider dapat menggunakan context tersebut.

13.6. Menggunakan Context Dengan useContext()

Setelah context dibuat dan Provider dipasang, kita dapat mengambil datanya menggunakan `useContext()`.

Misalnya pada `Header.jsx`:

```
import { useContext } from "react";
import { UserContext } from "../context/UserContext";

const Header = () => {

  const { user } = useContext(UserContext);

  return (
    <header>
      <h1>CareerHub</h1>

      <p>
        Halo, {user.name}
      </p>
    </header>
  );
};

export default Header;
```

Pada kode tersebut:

```
const { user } = useContext(UserContext);
```

digunakan untuk mengambil data user dari `UserContext`.

Hasilnya:

CareerHub

Halo, Wahyu

Kita tidak perlu mengirim:

```
<Header user={user} />
```

dari `App.jsx`.

Data dapat diambil langsung dari Context.

13.7. Mengubah Data Pada Context

Context tidak hanya dapat digunakan untuk membaca data. Kita juga dapat mengubah data yang disimpan di dalamnya.

Sebelumnya kita memiliki:

```
const [user, setUser] = useState({
  name: "Wahyu",
  email: "wahyu@example.com"
});
```

Karena setUser juga diberikan melalui:

```
value={{
  user,
  setUser
}}
```

maka komponen lain dapat menggunakannya.

Contohnya:

```
import { useContext } from "react";
import { UserContext } from "../context/UserContext";

const Profile = () => {

  const { user, setUser } =
    useContext(UserContext);

  const handleChangeName = () => {

    setUser({
      ...user,
      name: "Wahyu Developer"
    });

  };

  return (
    <div>

      <h2>{user.name}</h2>

      <p>{user.email}</p>

      <button
```

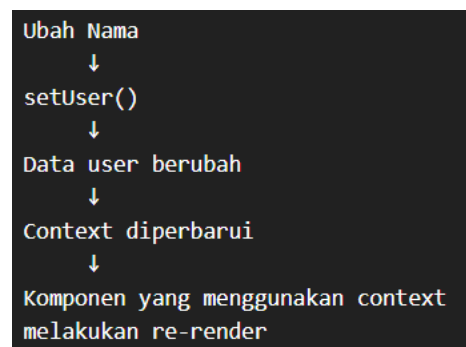
```

        onClick={handleChangeName}
        className="px-4 py-2 rounded-lg bg-blue-600 text-white"
      >
        Ubah Nama
      </button>
    </div>
  );
};

export default Profile;

```

Ketika tombol diklik:



13.8. Context API Pada CareeHub

Sekarang kita dapat melihat contoh penggunaan Context API pada CareerHub.

Misalnya CareerHub memiliki:

```

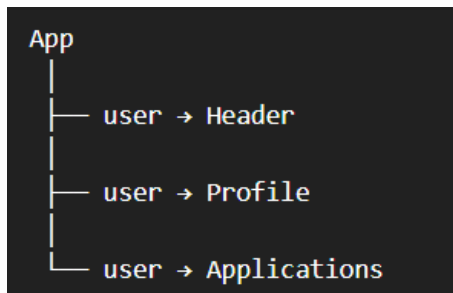
const user = {
  name: "Wahyu",
  email: "wahyu@example.com"
};

```

Data tersebut dibutuhkan oleh:

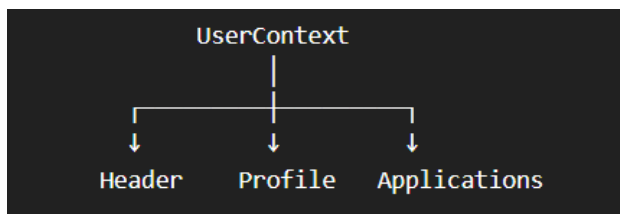
- Header
- Profile
- Applications

Tanpa Context API:



Jika struktur aplikasinya semakin kompleks, pengiriman props dapat menjadi lebih sulit.

Dengan Context API:



Setiap komponen dapat mengambil data secara langsung:

```
const { user } = useContext(UserContext);
```

Hal ini membuat pengelolaan data yang digunakan oleh banyak komponen menjadi lebih sederhana.

13.9. Context API Dan Props

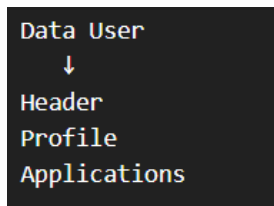
Context API bukan berarti props tidak diperlukan lagi. Props tetap digunakan ketika data hanya dibutuhkan oleh komponen tertentu.

Contohnya:

```
<JobCard job={job} />
```

Jika job hanya digunakan oleh JobCard, penggunaan props sudah cukup. Context lebih cocok digunakan ketika data perlu digunakan oleh banyak komponen yang berbeda.

Contohnya:



Maka Context dapat menjadi pilihan yang lebih tepat.

Secara sederhana:

Props	Context
Data dikirim dari parent ke child	Data dapat digunakan oleh banyak komponen
Cocok untuk data lokal	Cocok untuk data yang digunakan bersama
Dapat menyebabkan prop drilling jika terlalu dalam	Mengurangi prop drilling
Sangat umum digunakan	Digunakan ketika memang diperlukan

13.10. Ringkasan

Pada bab ini kita telah mempelajari:

- Pengertian Context API.
- Masalah Prop Drilling.
- Membuat Context menggunakan createContext().
- Membuat Provider.
- Memberikan data menggunakan value.
- Menggunakan Provider pada aplikasi CareerHub.
- Mengambil data Context menggunakan useContext().
- Mengubah data yang terdapat pada Context.
- Perbedaan penggunaan Props dan Context.
- Menggunakan Context untuk berbagi data antar komponen.

BAB 14

REACT ROUTER

14.1. Pendahuluan

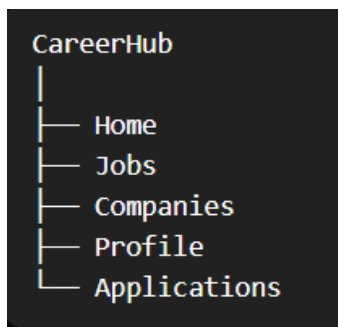
Pada bab sebelumnya kita telah mempelajari Context API yang dapat digunakan untuk berbagi data antar komponen dalam aplikasi React.

Saat ini aplikasi CareerHub sudah memiliki beberapa bagian, seperti:

- Halaman daftar lowongan
- Halaman detail lowongan
- Halaman perusahaan
- Halaman profile

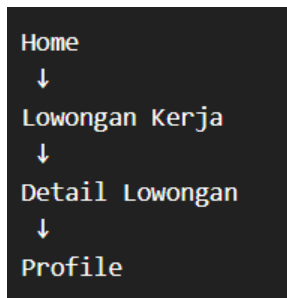
Namun, jika semua halaman dibuat dalam satu komponen App.jsx, struktur aplikasi akan menjadi semakin sulit untuk dikelola.

Contohnya:



Pengguna tentu membutuhkan cara untuk berpindah dari satu halaman ke halaman lainnya.

Misalnya:



Pada React, perpindahan halaman dapat dilakukan menggunakan React Router.

React Router memungkinkan kita membuat sistem navigasi pada aplikasi React sehingga setiap halaman memiliki URL masing-masing.

Contohnya:

```
/
```

untuk halaman Home.

```
/jobs
```

untuk halaman daftar lowongan.

```
/profile
```

untuk halaman Profile.

Dengan demikian, aplikasi CareerHub dapat memiliki beberapa halaman tanpa harus membuat aplikasi React yang berbeda untuk setiap halaman.

14.2. Apa Itu React Router

React Router adalah library yang digunakan untuk mengatur routing dan navigasi pada aplikasi React.

Routing menentukan halaman atau komponen apa yang ditampilkan berdasarkan URL yang sedang dibuka.

Contohnya:

URL	Halaman
/	Home
/jobs	Job List
/companies	Company List
/profile	Profile

Ketika pengguna membuka:

```
http://localhost:5173/jobs
```

React Router akan menampilkan halaman daftar lowongan.

Sedangkan ketika pengguna membuka:

```
http://localhost:5173/profile
```

React Router akan menampilkan halaman profile.

14.3. Instalasi React Router

Sebelum menggunakan React Router, kita perlu menginstalnya terlebih dahulu. Karena pada project kita menggunakan pnpm, jalankan perintah:

```
pnpm add react-router-dom
```

Setelah proses instalasi selesai, React Router sudah dapat digunakan pada project.

Struktur project dapat dibuat seperti:

```
src
├── components
│   ├── Header.jsx
│   └── JobCard.jsx
├── pages
│   ├── Home.jsx
│   ├── Jobs.jsx
│   ├── Profile.jsx
│   └── JobDetail.jsx
├── App.jsx
└── main.jsx
```

Kita dapat memisahkan komponen page dengan komponen biasa.

Contohnya:

```
pages
├── Home.jsx
├── Jobs.jsx
├── Profile.jsx
└── JobDetail.jsx
```

Sedangkan komponen yang digunakan berulang kali dapat diletakkan di:

```
components
├── Header.jsx
├── JobCard.jsx
└── Footer.jsx
```

Struktur tersebut membuat project lebih terorganisir.

14.4. Membuat Halaman CareerHub

Sebelum membuat routing, kita dapat membuat beberapa halaman sederhana.

Home.jsx

```
const Home = () => {
  return (
    <div>
      <h1>CareerHub</h1>

      <p>
        Temukan pekerjaan impianmu
      </p>
    </div>
  );
};

export default Home;
```

Jobs.jsx

```
const Jobs = () => {
  return (
    <div>
```

```

        <h1>Daftar Lowongan Kerja</h1>

        <p>
            Temukan berbagai lowongan kerja
            yang tersedia.
        </p>
    </div>
);
};

export default Jobs;

```

Profile.jsx

```

const Profile = () => {
    return (
        <div>
            <h1>Profile</h1>

            <p>
                Halaman profile pengguna.
            </p>
        </div>
    );
};

export default Profile;

```

Dengan demikian kita memiliki tiga halaman:

- Home
- Jobs
- Profile

Selanjutnya kita akan menghubungkan halaman tersebut menggunakan React Router.

14.5. Membuat Route

Untuk membuat route, kita dapat menggunakan beberapa komponen dari React Router:

```

import {
    BrowserRouter,
    Routes,

```

```
Route
} from "react-router-dom";
```

Masing-masing memiliki fungsi yang berbeda.

1. BrowserRouter

BrowserRouter digunakan sebagai pembungkus utama aplikasi agar fitur routing dapat digunakan.

2. Routes

Routes digunakan untuk menampung kumpulan route.

3. Route

Route digunakan untuk menentukan hubungan antara URL dengan component yang akan ditampilkan.

Contoh:

```
<BrowserRouter>

  <Routes>

    <Route
      path="/"
      element={<Home />}
    />

    <Route
      path="/jobs"
      element={<Jobs />}
    />

    <Route
      path="/profile"
      element={<Profile />}
    />

  </Routes>

</BrowserRouter>
```

Pada kode tersebut terdapat tiga route.

Route pertama:

```
<Route
```

```
    path="/"
    element={<Home />}
  />
```

Artinya ketika URL:

```
/
```

dibuka, React akan menampilkan:

```
<Home />
```

Route kedua:

```
<Route
  path="/jobs"
  element={<Jobs />}
/>
```

Artinya:

```
/jobs
```

akan menampilkan:

```
<Jobs />
```

Sedangkan:

```
<Route
  path="/profile"
  element={<Profile />}
/>
```

akan menampilkan:

```
<Profile />
```

14.6. Menerapkan Routing Pada App.jsx

Sekarang kita dapat menggabungkan seluruh route ke dalam App.jsx.

```
import {
  BrowserRouter,
  Routes,
  Route
```

```

} from "react-router-dom";

import Home from "../pages/Home";
import Jobs from "../pages/Jobs";
import Profile from "../pages/Profile";

const App = () => {

  return (
    <BrowserRouter>

      <Routes>

        <Route
          path="/"
          element={<Home />}
        />

        <Route
          path="/jobs"
          element={<Jobs />}
        />

        <Route
          path="/profile"
          element={<Profile />}
        />

      </Routes>

    </BrowserRouter>
  );
};

export default App;

```

Sekarang aplikasi memiliki tiga URL:

- <http://localhost:5173/>
- <http://localhost:5173/jobs>
- <http://localhost:5173/profile>

Masing-masing URL akan menampilkan halaman yang berbeda.

14.7. Membuat Navigasi Antar Halaman

Setelah route dibuat, pengguna membutuhkan menu untuk berpindah antar halaman. React Router menyediakan komponen:

Link

untuk melakukan navigasi.

Import:

```
import { Link } from "react-router-dom";
```

Kemudian kita dapat membuat navigasi:

```
<nav>

  <Link to="/">
    Home
  </Link>

  <Link to="/jobs">
    Lowongan Kerja
  </Link>

  <Link to="/profile">
    Profile
  </Link>

</nav>
```

Ketika pengguna mengklik:

Lowongan Kerja

maka React akan mengarahkan pengguna ke:

/jobs

Sedangkan ketika pengguna mengklik:

Profile

maka pengguna diarahkan ke:

/profile

14.8. Membuat Header CareerHub

Kita dapat menggunakan konsep tersebut pada Header.jsx.

```
import { Link } from "react-router-dom";

const Header = () => {

  return (
    <header className="border-b">

      <div className="max-w-7xl mx-auto px-6 py-4 flex justify-between">

        <Link
          to="/"
          className="text-2xl font-bold"
        >
          CareerHub
        </Link>

        <nav className="flex gap-6">

          <Link to="/">
            Home
          </Link>

          <Link to="/jobs">
            Lowongan Kerja
          </Link>

          <Link to="/profile">
            Profile
          </Link>

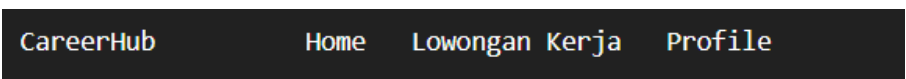
        </nav>

      </div>

    </header>
  );
};

export default Header;
```

Hasilnya kurang lebih:



CareerHub Home Lowongan Kerja Profile

Ketika pengguna mengklik Lowongan Kerja, URL berubah menjadi:

`/jobs`

dan halaman Jobs ditampilkan.

14.9. Menempatkan Header Pada Semua Halaman

Biasanya Header ingin ditampilkan pada seluruh halaman CareerHub. Kita dapat menempatkan Header di luar Routes.

```
import {
  BrowserRouter,
  Routes,
  Route
} from "react-router-dom";

import Header from "../components/Header";

import Home from "../pages/Home";
import Jobs from "../pages/Jobs";
import Profile from "../pages/Profile";

const App = () => {
  return (
    <BrowserRouter>

      <Header />

      <main className="max-w-7xl mx-auto px-6 py-8">

        <Routes>

          <Route
            path="/"
            element={<Home />}
          />

          <Route
            path="/jobs"
            element={<Jobs />}
          />

          <Route
```

```

        path="/profile"
        element={<Profile />}
      />

    </Routes>

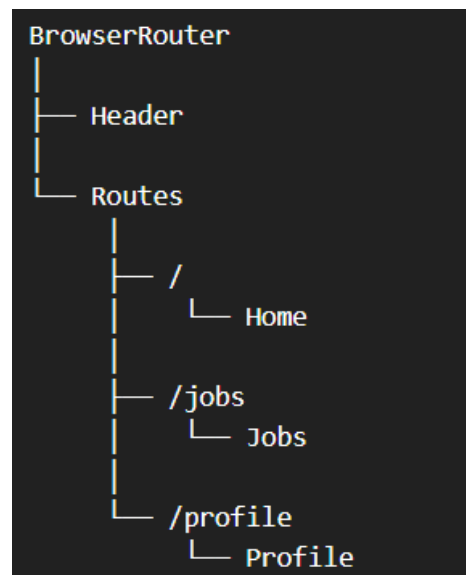
  </main>

</BrowserRouter>
);
};

export default App;

```

Struktur aplikasinya menjadi:



Dengan struktur tersebut, Header akan tetap tampil ketika pengguna berpindah halaman.

14.10. Link Vs <a>

Dalam aplikasi React yang menggunakan React Router, navigasi antar halaman sebaiknya menggunakan:

```

<Link to="/jobs">
  Lowongan Kerja
</Link>

```

bukan:

```
<a href="/jobs">
  Lowongan Kerja
</a>
```

Link dirancang untuk digunakan oleh React Router sehingga navigasi dapat dilakukan tanpa melakukan reload halaman secara penuh.

Contohnya:

```
<Link to="/profile">
  Profile
</Link>
```

Ketika diklik, React Router akan menangani perpindahan route.

Sedangkan penggunaan:

```
<a href="/profile">
  Profile
</a>
```

akan menggunakan navigasi browser biasa.

Oleh karena itu, untuk perpindahan halaman di dalam aplikasi React, gunakan Link.

14.11. Menambahkan Halaman Detail Lowongan

CareerHub sebelumnya sudah memiliki data lowongan seperti:

```
const jobs = [
  {
    id: 1,
    jobName: "Fullstack Developer",
    company: "PT. ABC Indonesia",
    location: "Bandung",
    salary: "Rp. 8.000.000",
    status: true
  },
  {
    id: 2,
    jobName: "Backend Developer",
    company: "PT. DEF Indonesia",
    location: "Jakarta",
    salary: "Rp. 8.000.000",
    status: false
  }
]
```

```
}  
];
```

Kita dapat membuat halaman detail:

```
/jobs/1
```

untuk menampilkan detail lowongan dengan ID 1.

Untuk membuat route tersebut, kita dapat menggunakan parameter:

```
<Route  
  path="/jobs/:id"  
  element={<JobDetail />}  
</>
```

Tanda : menunjukkan bahwa bagian tersebut merupakan parameter.

Contohnya:

- /jobs/1
- /jobs/2
- /jobs/3

Semua URL tersebut menggunakan route:

```
/jobs/:id
```

Nilai id dapat berbeda-beda.

14.12. Navigasi Dari Job Card Ke Detail

Kita dapat menggunakan Link pada JobCard.

```
import { Link } from "react-router-dom";  
  
const JobCard = ({ job }) => {  
  return (  
    <div className="border rounded-xl p-5">  
      <h3 className="text-xl font-bold">  
        {job.jobName}  
      </h3>
```

```

        <p>{job.company}</p>

        <p>{job.location}</p>

        <p>{job.salary}</p>

        <Link
          to={` /jobs/${job.id}`}
          className="inline-block mt-4 px-4 py-2 bg-blue-600 text-white
rounded-lg"
        >
          Lihat Detail
        </Link>

      </div>
    );
  };
export default JobCard;

```

Jika:

```
job.id = 1
```

maka:

```
to={` /jobs/${job.id}`}
```

akan menghasilkan:

```
/jobs/1
```

Jika:

```
job.id = 2
```

maka menghasilkan:

```
/jobs/2
```

Dengan demikian, setiap lowongan dapat memiliki halaman detail masing-masing.

14.13. Mengambil Parameter URL

Pada halaman JobDetail, kita dapat mengambil nilai id menggunakan:

useParams()

Contoh:

```
import { useParams } from "react-router-dom";

const JobDetail = () => {

  const { id } = useParams();

  return (
    <div>

      <h1>Detail Lowongan</h1>

      <p>
        ID Lowongan: {id}
      </p>

    </div>
  );
};

export default JobDetail;
```

Jika pengguna membuka:

/jobs/1

maka:

id

akan bernilai:

1

Jika membuka:

/jobs/2

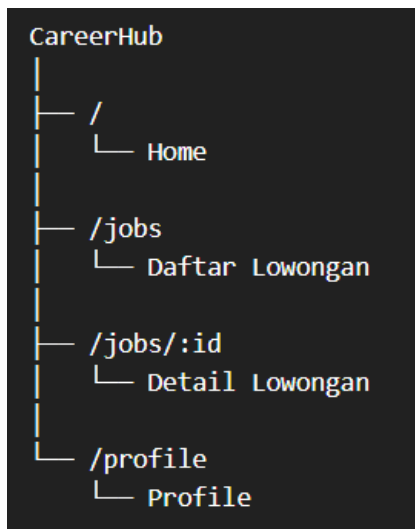
maka:

id = 2

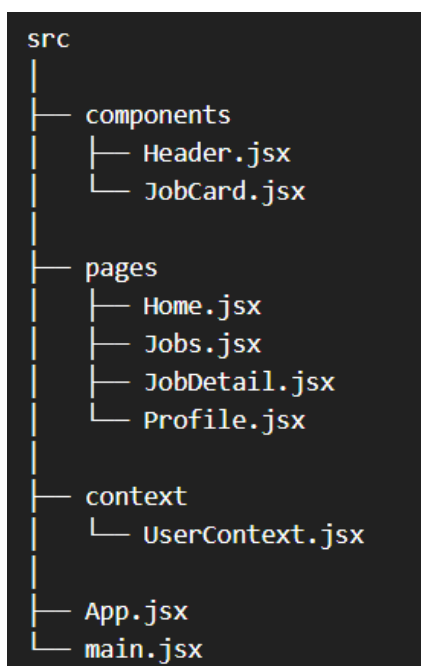
Konsep ini akan sangat berguna ketika kita ingin mengambil data berdasarkan ID.

14.14. Struktur Routing CareerHub

Setelah menerapkan React Router, struktur halaman CareerHub dapat menjadi:



Sedangkan struktur folder:



Dengan demikian, aplikasi CareerHub sudah memiliki struktur navigasi yang lebih terorganisir.

14.15. Ringkasan

Pada bab ini kita telah mempelajari:

- Pengertian React Router.
- Instalasi React Router.
- Membuat BrowserRouter.
- Membuat Routes dan Route.
- Menentukan URL menggunakan path.
- Menampilkan halaman menggunakan element.
- Membuat navigasi menggunakan Link.
- Membuat navigasi antar halaman CareerHub.
- Membuat route dengan parameter menggunakan :id.
- Mengambil parameter URL menggunakan useParams().

BAB 15

DEPLOYMENT

15.1. Pendahuluan

Pada bab sebelumnya kita telah mempelajari Routing dengan React Router untuk membuat navigasi antar halaman pada aplikasi CareerHub. Selama proses pengembangan, aplikasi biasanya dijalankan menggunakan:

```
pnpm dev
```

Kemudian aplikasi dapat diakses melalui:

```
http://localhost:5173
```

Namun, alamat tersebut hanya dapat digunakan pada komputer yang sedang menjalankan project. Agar aplikasi CareerHub dapat diakses oleh orang lain melalui internet, aplikasi perlu didploy ke sebuah platform hosting.

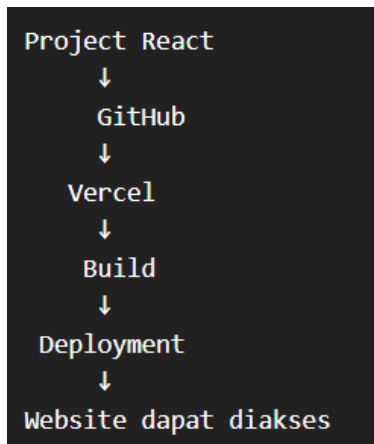
Salah satu platform yang dapat digunakan untuk melakukan deployment aplikasi React adalah Vercel. Pada bab ini kita akan mempelajari pengenalan Vercel dan langkah-langkah melakukan deployment aplikasi React berbasis Vite ke Vercel.

15.2. Apa Itu Vercel

Vercel adalah platform yang dapat digunakan untuk melakukan deployment dan hosting aplikasi web. Vercel banyak digunakan untuk aplikasi frontend seperti React, Next.js, dan berbagai project berbasis JavaScript.

Untuk project React menggunakan Vite, Vercel dapat mendeteksi project dan melakukan proses build untuk menghasilkan aplikasi yang siap digunakan.

Secara sederhana, prosesnya:



Setelah berhasil dideploy, Vercel akan memberikan URL untuk mengakses aplikasi.

Contohnya:

`https://careerhub.vercel.app`

Vercel juga mendukung deployment melalui Git. Ketika repository sudah terhubung, perubahan pada repository dapat memicu deployment baru secara otomatis.

15.3. Mengapa Menggunakan Vercel

Vercel memiliki beberapa kelebihan untuk deployment aplikasi React.

1) Mudah Digunakan

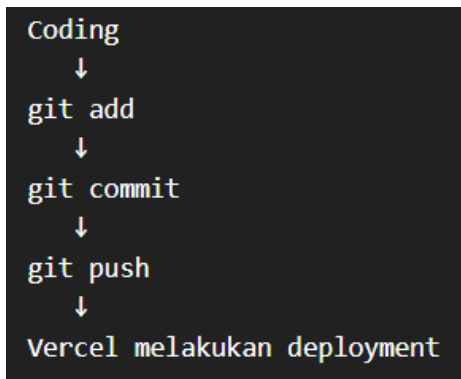
Project dapat dihubungkan langsung dengan repository GitHub.

Kita tidak perlu melakukan konfigurasi server secara manual untuk deployment frontend sederhana.

2) Terintegrasi dengan Git

Vercel dapat terhubung dengan GitHub, GitLab, Bitbucket, dan beberapa provider Git lainnya.

Dengan demikian, ketika terdapat perubahan pada project:



3) Mendukung Preview Deployment

Vercel dapat membuat deployment preview untuk perubahan yang sedang dikembangkan. Hal ini memungkinkan kita melihat hasil perubahan sebelum menjadikannya deployment production.

4) Mendapatkan URL Online

Setelah deployment berhasil, aplikasi dapat diakses melalui URL yang diberikan oleh Vercel. Vercel secara otomatis memberikan domain dengan format `.vercel.app` untuk deployment React.

15.4. Persiapan Sebelum Deployment

Sebelum melakukan deployment CareerHub, pastikan project dapat berjalan dengan baik di komputer.

Jalankan:

```
pnpm install
```

Kemudian:

```
pnpm dev
```

Buka:

```
http://localhost:5173
```

Pastikan aplikasi dapat berjalan tanpa error.

Selain itu, kita juga sebaiknya melakukan build production terlebih dahulu.

```
pnpm build
```

Jika proses berhasil, Vite akan menghasilkan folder:

```
dist
```

Folder tersebut berisi hasil build aplikasi yang siap untuk digunakan pada production.

Secara umum:

```
Project React
├── src
├── public
├── package.json
├── vite.config.js
└── dist
```

Folder dist dihasilkan dari proses:

```
pnpm build
```

15.5. Menyimpan Project Ke Github

Salah satu cara yang paling mudah untuk melakukan deployment ke Vercel adalah melalui repository Git.

Pastikan project CareerHub sudah memiliki repository GitHub.

Jika project belum menggunakan Git, jalankan:

```
git init
```

Kemudian:

```
git add .
```

Buat commit:

```
git commit -m "Initial CareerHub project"
```

Hubungkan dengan repository GitHub:

```
git remote add origin https://github.com/username/careerhub.git
```

Kemudian push:

```
git branch -M main  
git push -u origin main
```

Setelah berhasil, project akan tersedia di GitHub.

Contohnya:

<https://github.com/username/careerhub>

15.6. Membuat Akun Vercel

Selanjutnya buka:

<https://vercel.com/>

Kemudian pilih opsi untuk melakukan login atau sign up.

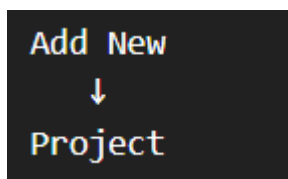
Untuk deployment menggunakan repository GitHub, kita dapat menghubungkan akun Vercel dengan GitHub.

Dengan koneksi tersebut, Vercel dapat mengakses repository yang akan digunakan untuk deployment. Vercel menyediakan proses import repository langsung dari dashboard.

15.7. Membuat Project Di Vercel

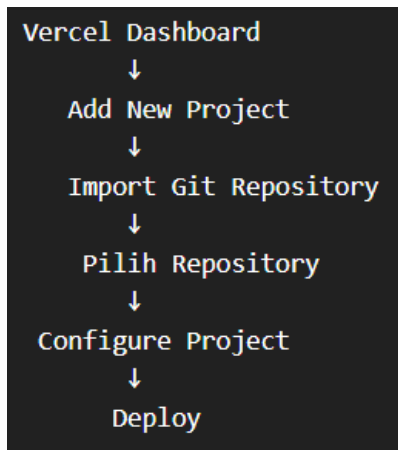
Setelah login, masuk ke dashboard Vercel.

Kemudian pilih:



Pada halaman pembuatan project, pilih repository GitHub yang berisi aplikasi CareerHub.

Secara umum prosesnya:

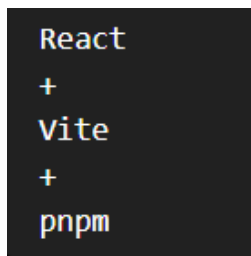


Vercel menyediakan pengaturan seperti nama project, framework, root directory, build settings, dan environment variables sebelum deployment dilakukan.

15.8. Mengatur Project React

Setelah repository dipilih, Vercel akan menampilkan konfigurasi project.

Untuk project CareerHub yang menggunakan:



Vercel biasanya dapat mendeteksi framework yang digunakan.

Periksa bagian:

Project Name

Misalnya:

careerhub

Framework Preset

Pilih:

Vite

atau gunakan framework yang terdeteksi otomatis oleh Vercel.

Vercel memang menyediakan dukungan untuk project Vite dan dapat melakukan deployment project Vite dari repository maupun melalui CLI.

Root Directory

Jika project berada langsung di repository:

```
/
```

maka tidak perlu mengubah root directory.

Contohnya:

```
careerhub
├──
├── src
├── public
├── package.json
└── vite.config.js
```

Namun jika project berada di dalam folder tertentu:

```
repository
├──
└── careerhub
    ├── src
    ├── package.json
    └── vite.config.js
```

maka Root Directory perlu diarahkan ke:

```
careerhub
```

15.9. Build Command Dan Output Directory

Pada project Vite, proses build biasanya menggunakan:

```
pnpm build
```

yang pada akhirnya menjalankan script build dari package.json.

Contohnya:

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

Hasil build Vite berada pada:

dist

Sehingga konfigurasi secara konsep:

```
Build Command
pnpm build

Output Directory
dist
```

Namun jika Vercel sudah mendeteksi konfigurasi tersebut secara otomatis, kita dapat menggunakan konfigurasi default yang diberikan Vercel.

15.10. Melakukan Deployment

Setelah konfigurasi selesai, klik:

Deploy

Vercel kemudian akan melakukan beberapa proses:

```
Repository
↓
Install Dependencies
↓
Build Project
↓
Generate Deployment
↓
Deployment Berhasil
```

Jika proses build berhasil, Vercel akan menampilkan informasi deployment beserta URL aplikasi.

Contohnya:

```
https://careerhub.vercel.app
```

Deployment Vercel menghasilkan URL unik yang dapat digunakan untuk melihat aplikasi yang sudah online.

15.11. Production Dan Preview Deployment

Vercel memiliki konsep Production Deployment dan Preview Deployment. Misalnya repository memiliki branch:

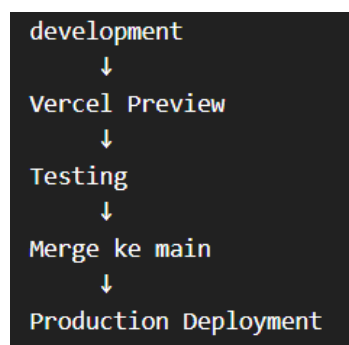
```
main
```

```
development
```

Branch main dapat digunakan sebagai production branch.

Sedangkan perubahan pada branch lain dapat digunakan untuk preview.

Contohnya:



Vercel membuat deployment preview untuk perubahan pada branch atau pull request sehingga perubahan dapat diperiksa sebelum masuk ke production.

Konsep ini sangat berguna ketika project dikerjakan secara tim.

15.12. Deployment Menggunakan Vercel CLI

Selain menggunakan dashboard dan GitHub, Vercel juga menyediakan Vercel CLI.

Untuk menginstalnya:

```
pnpm add -g vercel
```

Setelah terinstall, masuk ke folder project:

```
cd careerhub
```

Kemudian jalankan:

```
vercel
```

Vercel CLI akan melakukan proses deployment dari folder project. Dokumentasi Vercel juga menyediakan perintah vercel untuk melakukan deployment dari root project. Untuk melakukan production deployment secara langsung:

```
vercel --prod
```

Perintah tersebut dapat digunakan untuk membuat Production Deployment dari project lokal.

Namun untuk pembelajaran CareerHub, metode GitHub + Vercel Dashboard lebih disarankan karena lebih mudah dipahami dan sekaligus memperkenalkan konsep deployment berbasis Git.

15.13. Masalah Yang Sering Terjadi

Setelah deployment, terkadang aplikasi tidak langsung berjalan dengan baik.

Beberapa masalah yang umum terjadi adalah:

1) Build Error

Misalnya terdapat error JavaScript atau dependency yang bermasalah.

Solusinya adalah mencoba build terlebih dahulu secara lokal:

```
pnpm build
```

Jika terdapat error, perbaiki error tersebut sebelum melakukan deployment ulang.

2) Halaman Tidak Ditemukan Setelah Refresh

CareerHub pada bab sebelumnya menggunakan React Router.

Misalnya pengguna membuka:

```
/jobs
```

Navigasi dari aplikasi berjalan dengan baik.

Namun setelah browser di-refresh, halaman dapat mengalami masalah jika konfigurasi hosting belum menangani routing SPA dengan benar.

Hal ini perlu diperhatikan terutama ketika menggunakan route seperti:

```
/jobs
```

```
/jobs/1
```

```
/profile
```

Jika masalah tersebut muncul, konfigurasi deployment perlu disesuaikan agar request route aplikasi diarahkan ke entry point React.

3) Environment Variable

Jika aplikasi menggunakan API, biasanya terdapat environment variable.

Contohnya:

```
VITE_API_URL
```

Environment variable tersebut perlu dikonfigurasi pada project Vercel jika dibutuhkan oleh aplikasi production.

Vercel menyediakan pengaturan Environment Variables pada konfigurasi project.

15.14. Ringkasan

Pada bab ini kita telah mempelajari:

- Pengertian Vercel.
- Fungsi Vercel untuk deployment aplikasi React.
- Persiapan aplikasi sebelum deployment.
- Menyimpan project React ke GitHub.

- Membuat akun Vercel.
- Menghubungkan repository GitHub dengan Vercel.
- Membuat project pada Vercel.
- Mengatur konfigurasi project React.
- Melakukan deployment aplikasi React ke Vercel.
- Perbedaan Production dan Preview Deployment.